

```

import csv
import os

# -----
# STEP 1: Create the training dataset (ws.csv)
# -----

csv_data = [
    ["Sunny", "Warm", "Normal", "Strong", "Warm", "Same", "Yes"],
    ["Sunny", "Warm", "High", "Strong", "Warm", "Same", "Yes"],
    ["Rainy", "Cold", "High", "Strong", "Warm", "Change", "No"],
    ["Sunny", "Warm", "High", "Strong", "Cool", "Change", "Yes"]
]

# Create ws.csv if it does not already exist
if not os.path.exists("ws.csv"):
    with open("ws.csv", "w", newline="") as csv_file:
        writer = csv.writer(csv_file)
        writer.writerows(csv_data)

# -----
# STEP 2: Initialize hypothesis
# -----

hypothesis = ['0'] * 6

# -----
# STEP 3: Read the CSV file
# -----

with open("ws.csv", "r", newline="") as csv_file:
    readcsv = csv.reader(csv_file, delimiter=',')

    data = []

    print("\nThe given training examples are:")

    # Read every training example
    for row in readcsv:
        print(row)

        # Select only positive examples
        if row[-1].strip().lower() == "yes":
            data.append(row)

# -----
# STEP 4: Display positive examples
# -----

print("\nThe positive examples are:")

for x in data:
    print(x)

# -----
# STEP 5: Find-S Algorithm
# -----

TotalExamples = len(data)

if TotalExamples > 0:

    # Number of attributes
    d = len(data[0]) - 1

    # Initialize hypothesis with the first positive example
    hypothesis = data[0][:d]

    print("\nThe steps of the Find-S algorithm are:")
    print(hypothesis)

    # Compare remaining positive examples
    for i in range(1, TotalExamples):

        for k in range(d):

            if hypothesis[k] != data[i][k]:
                hypothesis[k] = '?'

    print(hypothesis)

```

```

        print(
            f"After training example {i + 1}, "
            f"hypothesis is: {hypothesis}"
        )

# -----
# STEP 6: Final hypothesis
# -----

print(
    "\nThe maximally specific Find-S hypothesis "
    "for the given training examples is:"
)

print(hypothesis)

else:
    print("\nNo positive examples found.")

```

The given training examples are:

```

['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes']
['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes']
['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No']
['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']

```

The positive examples are:

```

['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes']
['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes']
['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']

```

The steps of the Find-S algorithm are:

```

['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']

```

After training example 2, hypothesis is: ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']

After training example 3, hypothesis is: ['Sunny', 'Warm', '?', 'Strong', '?', '?']

The maximally specific Find-S hypothesis for the given training examples is:

```

['Sunny', 'Warm', '?', 'Strong', '?', '?']

```

experiment 2

```

import numpy as np
import pandas as pd
import os

# -----
# STEP 1: Create the training dataset
# -----

csv_data = [
    ["Sunny", "Warm", "Normal", "Strong", "Warm", "Same", "Yes"],
    ["Sunny", "Warm", "High", "Strong", "Warm", "Same", "Yes"],
    ["Rainy", "Cold", "High", "Strong", "Warm", "Change", "No"],
    ["Sunny", "Warm", "High", "Strong", "Cool", "Change", "Yes"]
]

columns = [
    "Sky",
    "AirTemp",
    "Humidity",
    "Wind",
    "Water",
    "Forecast",
    "EnjoySport"
]

# Create CSV file if it does not exist
file_name = "trainingdata_candidate_elimination.csv"

if not os.path.exists(file_name):
    df = pd.DataFrame(csv_data, columns=columns)
    df.to_csv(file_name, index=False)

# -----
# STEP 2: Read the training data
# -----

data = pd.read_csv(file_name)

print("The given training examples are:")
print(data)

# -----

```

```

# STEP 3: Separate concepts and target
# -----

concepts = np.array(data.iloc[:, 0:-1])

print("\nConcepts:")
print(concepts)

target = np.array(data.iloc[:, -1])

print("\nTarget:")
print(target)

# -----
# STEP 4: Candidate Elimination Algorithm
# -----

def learn(concepts, target):

    # Initial specific hypothesis
    specific_h = concepts[0].copy()

    print("\nInitialization of specific_h and general_h")

    print("Specific Hypothesis:")
    print(specific_h)

    # Initial general hypothesis
    general_h = [
        ['?' for _ in range(len(specific_h))]
        for _ in range(len(specific_h))
    ]

    print("General Hypothesis:")
    print(general_h)

    # -----
    # Process each training example
    # -----

    for i, h in enumerate(concepts):

        print(f"\nInstance {i + 1}: {h}")
        print("Target:", target[i])

        # -----
        # Positive example
        # -----

        if target[i] == "Yes":

            for x in range(len(specific_h)):

                if h[x] != specific_h[x]:

                    specific_h[x] = '?'

                    general_h[x][x] = '?'

        # -----
        # Negative example
        # -----

        elif target[i] == "No":

            for x in range(len(specific_h)):

                if h[x] != specific_h[x]:

                    general_h[x][x] = specific_h[x]

                else:

                    general_h[x][x] = '?'

        # -----
        # Display intermediate hypotheses
        # -----

    print(
        "Specific Hypothesis after instance",

```

```

        i + 1,
        ":",
        specific_h
    )

    print(
        "General Hypothesis after instance",
        i + 1,
        ":",
        general_h
    )

# -----
# Remove completely general rows
# -----

general_h = [
    h for h in general_h
    if any(attr != '?' for attr in h)
]

return specific_h, general_h

# -----
# STEP 5: Run Candidate Elimination
# -----

s_final, g_final = learn(concepts, target)

# -----
# STEP 6: Display final hypotheses
# -----

print("\nFinal Specific Hypothesis:")
print(s_final)

print("\nFinal General Hypothesis:")
print(g_final)

```

The given training examples are:

	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
0	Sunny	Warm	Normal	Strong	Warm	Same	Yes
1	Sunny	Warm	High	Strong	Warm	Same	Yes
2	Rainy	Cold	High	Strong	Warm	Change	No
3	Sunny	Warm	High	Strong	Cool	Change	Yes

Concepts:

```

[['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']
 ['Sunny' 'Warm' 'High' 'Strong' 'Warm' 'Same']
 ['Rainy' 'Cold' 'High' 'Strong' 'Warm' 'Change']
 ['Sunny' 'Warm' 'High' 'Strong' 'Cool' 'Change']]

```

Target:

```
['Yes' 'Yes' 'No' 'Yes']
```

Initialization of specific_h and general_h

Specific Hypothesis:

```
['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']
```

General Hypothesis:

```
[[ '?', '?', '?', '?', '?', '?' ], [ '?', '?', '?', '?', '?', '?' ], [ '?', '?', '?', '?', '?', '?' ], [ '?', '?', '?', '?', '?', '?' ],
```

Instance 1: ['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']

Target: Yes

Specific Hypothesis after instance 1 : ['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']

General Hypothesis after instance 1 : [['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'],

Instance 2: ['Sunny' 'Warm' 'High' 'Strong' 'Warm' 'Same']

Target: Yes

Specific Hypothesis after instance 2 : ['Sunny' 'Warm' '?' 'Strong' 'Warm' 'Same']

General Hypothesis after instance 2 : [['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'],

Instance 3: ['Rainy' 'Cold' 'High' 'Strong' 'Warm' 'Change']

Target: No

Specific Hypothesis after instance 3 : ['Sunny' 'Warm' '?' 'Strong' 'Warm' 'Same']

General Hypothesis after instance 3 : [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'],

Instance 4: ['Sunny' 'Warm' 'High' 'Strong' 'Cool' 'Change']

Target: Yes

Specific Hypothesis after instance 4 : ['Sunny' 'Warm' '?' 'Strong' '?' '?']

General Hypothesis after instance 4 : [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'],

Final Specific Hypothesis:

```
['Sunny' 'Warm' '?' 'Strong' '?' '?']
```

```
Final General Hypothesis:
[['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?']]
```

Experiment 3

```
import math
import csv
import os

# =====
# STEP 1: CREATE TRAINING DATASET
# =====

training_data = [
    ["Outlook", "Temperature", "Humidity", "Wind", "Play"],
    ["Sunny", "Hot", "High", "Weak", "No"],
    ["Sunny", "Hot", "High", "Strong", "No"],
    ["Overcast", "Hot", "High", "Weak", "Yes"],
    ["Rain", "Mild", "High", "Weak", "Yes"],
    ["Rain", "Cool", "Normal", "Weak", "Yes"],
    ["Rain", "Cool", "Normal", "Strong", "No"],
    ["Overcast", "Cool", "Normal", "Strong", "Yes"],
    ["Sunny", "Mild", "High", "Weak", "No"],
    ["Sunny", "Cool", "Normal", "Weak", "Yes"],
    ["Rain", "Mild", "Normal", "Weak", "Yes"],
    ["Sunny", "Mild", "Normal", "Strong", "Yes"],
    ["Overcast", "Mild", "High", "Strong", "Yes"],
    ["Overcast", "Hot", "Normal", "Weak", "Yes"],
    ["Rain", "Mild", "High", "Strong", "No"]
]

test_data = [
    ["Sunny", "Cool", "High", "Strong"],
    ["Overcast", "Mild", "High", "Weak"],
    ["Rain", "Cool", "Normal", "Strong"]
]

# Create data3.csv
if not os.path.exists("data3.csv"):

    with open("data3.csv", "w", newline="") as file:

        writer = csv.writer(file)

        writer.writerows(training_data)

# Create data3_test.csv
if not os.path.exists("data3_test.csv"):

    with open("data3_test.csv", "w", newline="") as file:

        writer = csv.writer(file)

        writer.writerows(test_data)

# =====
# STEP 2: LOAD CSV FILE
# =====

def load_csv(filename):

    with open(filename, "r", newline="") as file:

        lines = csv.reader(file)

        dataset = list(lines)

        headers = dataset.pop(0)

        return dataset, headers

# =====
# STEP 3: NODE CLASS
# =====
```

```

class Node:

    def __init__(self, attribute):

        self.attribute = attribute

        self.children = []

        self.answer = ""

# =====
# STEP 4: CREATE SUBTABLES
# =====

def subtables(data, col, delete):

    dic = {}

    coldata = [row[col] for row in data]

    attr = list(set(coldata))

    counts = [0] * len(attr)

    r = len(data)

    c = len(data[0])

    # Count occurrences
    for x in range(len(attr)):

        for y in range(r):

            if data[y][col] == attr[x]:

                counts[x] += 1

    # Create dictionary
    for x in range(len(attr)):

        dic[attr[x]] = [
            [0 for i in range(c)]
            for j in range(counts[x])
        ]

    # Fill dictionary
    for x in range(len(attr)):

        pos = 0

        for y in range(r):

            if data[y][col] == attr[x]:

                row = data[y].copy()

                if delete:

                    del row[col]

                dic[attr[x]][pos] = row

                pos += 1

    return attr, dic

# =====
# STEP 5: ENTROPY
# =====

def entropy(S):

    attr = list(set(S))

    if len(attr) == 1:

```

```

        return 0

counts = [0] * len(attr)

total = len(S)

for i in range(len(attr)):

    counts[i] = sum(
        1 for x in S
        if attr[i] == x
    ) / (total * 1.0)

sums = 0

for cnt in counts:

    if cnt != 0:

        sums += -1 * cnt * math.log(cnt, 2)

return sums

# =====
# STEP 6: COMPUTE INFORMATION GAIN
# =====

def compute_gain(data, col):

    attr, dic = subtables(
        data,
        col,
        delete=False
    )

    total_size = len(data)

    entropies = [0] * len(attr)

    ratio = [0] * len(attr)

    total_entropy = entropy(
        [row[-1] for row in data]
    )

    for x in range(len(attr)):

        ratio[x] = (
            len(dic[attr[x]])
            /
            (total_size * 1.0)
        )

        entropies[x] = entropy(
            [row[-1] for row in dic[attr[x]]]
        )

        total_entropy -= (
            ratio[x] * entropies[x]
        )

    return total_entropy

# =====
# STEP 7: BUILD DECISION TREE
# =====

def build_tree(data, features):

    lastcol = [row[-1] for row in data]

    # If all output values are same
    if len(set(lastcol)) == 1:

```

```

        node = Node("")

        node.answer = lastcol[0]

        return node

# Number of attributes
n = len(data[0]) - 1

gains = [0] * n

# Calculate information gain
for col in range(n):

    gains[col] = compute_gain(
        data,
        col
    )

# Select attribute with maximum gain
split = gains.index(
    max(gains)
)

node = Node(
    features[split]
)

# Remove selected feature
fea = (
    features[:split]
    +
    features[split + 1:]
)

# Create subtables
attr, dic = subtables(
    data,
    split,
    delete=True
)

# Create child nodes
for x in range(len(attr)):

    child = build_tree(
        dic[attr[x]],
        fea
    )

    node.children.append(
        (attr[x], child)
    )

return node

# =====
# STEP 8: PRINT DECISION TREE
# =====

def print_tree(node, level):

    if node.answer != "":

        print(
            " " * level,
            "->",
            node.answer
        )

    return

```



```

    print(
        " " * level,
        node.attribute
    )

    for value, n in node.children:

        print(
            " " * (level + 1),
            value
        )

        print_tree(
            n,
            level + 2
        )

# =====
# STEP 9: CLASSIFY TEST INSTANCE
# =====

def classify(node, x_test, features):

    if node.answer != "":

        print(node.answer)

        return

    pos = features.index(
        node.attribute
    )

    for value, n in node.children:

        if x_test[pos] == value:

            classify(
                n,
                x_test,
                features
            )

            return

    print("Unknown")

# =====
# MAIN PROGRAM
# =====

dataset, features = load_csv(
    "data3.csv"
)

# Build ID3 tree
node1 = build_tree(
    dataset,
    features
)

print(
    "\nThe decision tree for the dataset using ID3 algorithm is:"
)

print_tree(
    node1,
    0
)

# =====
# TEST DATA

```

```
# =====

testdata, test_features = load_csv(
    "data3_test.csv"
)

print("\nTest Results:")

for xtest in testdata:

    print(
        "\nThe test instance:",
        xtest
    )

    print(
        "The label for test instance:",
        end=" "
    )

    classify(
        node1,
        xtest,
        features
    )
```

The decision tree for the dataset using ID3 algorithm is:

```
Outlook
Overcast
-> Yes
Rain
Wind
Weak
-> Yes
Strong
-> No
Sunny
Humidity
High
-> No
Normal
-> Yes
```

Test Results:

```
The test instance: ['Overcast', 'Mild', 'High', 'Weak']
The label for test instance: Yes
```

```
The test instance: ['Rain', 'Cool', 'Normal', 'Strong']
The label for test instance: No
```

Experiment 4

```
import numpy as np

# -----
# INPUT DATA
# -----

X = np.array(
    ([2, 9],
     [1, 5],
     [3, 6]),
    dtype=float
)

# Target output
y = np.array(
    ([92],
     [86],
     [89]),
    dtype=float
)

# -----
# NORMALIZATION
# -----

# Normalize X
```

```

X = X / np.amax(X, axis=0)

# Normalize y
y = y / 100

# -----
# SIGMOID FUNCTION
# -----

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# -----
# DERIVATIVE OF SIGMOID
# -----

def derivatives_sigmoid(x):
    return x * (1 - x)

# -----
# PARAMETERS
# -----

epoch = 5

lr = 0.1

inputlayer_neurons = 2

hiddenlayer_neurons = 3

output_neurons = 1

# -----
# INITIALIZE WEIGHTS AND BIASES
# -----

wh = np.random.uniform(
    size=(inputlayer_neurons, hiddenlayer_neurons)
)

bh = np.random.uniform(
    size=(1, hiddenlayer_neurons)
)

wout = np.random.uniform(
    size=(hiddenlayer_neurons, output_neurons)
)

bout = np.random.uniform(
    size=(1, output_neurons)
)

# -----
# TRAINING
# -----

for i in range(epoch):

    # =====
    # FORWARD PROPAGATION
    # =====

    # Input to hidden layer
    hinp1 = np.dot(X, wh)

    hinp = hinp1 + bh

    # Hidden layer activation
    hlayer_act = sigmoid(hinp)

    # Hidden layer to output layer
    outinp1 = np.dot(hlayer_act, wout)

    outinp = outinp1 + bout

    # Output activation

```

```

output = sigmoid(outinp)

# =====
# BACKPROPAGATION
# =====

# Error in output
EO = y - output

# Gradient at output layer
outgrad = derivatives_sigmoid(output)

d_output = EO * outgrad

# Error at hidden layer
EH = d_output.dot(wout.T)

# Gradient at hidden layer
hiddengrad = derivatives_sigmoid(hlayer_act)

d_hiddenlayer = EH * hiddengrad

# =====
# UPDATE WEIGHTS
# =====

wout += hlayer_act.T.dot(d_output) * lr
wh += X.T.dot(d_hiddenlayer) * lr

# =====
# DISPLAY OUTPUT
# =====

print(
    "-----Epoch-",
    i + 1,
    "Starts-----"
)

print("Input: \n" + str(X))

print("Actual Output: \n" + str(y))

print(
    "Predicted Output: \n",
    output
)

print(
    "-----Epoch-",
    i + 1,
    "Ends-----\n"
)

# -----
# FINAL OUTPUT
# -----

print("=====")

print("Final Input: \n" + str(X))

print("Final Actual Output: \n" + str(y))

print("Final Predicted Output: \n", output)

print("=====")

```

```

-----Epoch- 1 Starts-----
Input:
[[0.66666667 1.          ]
 [0.33333333 0.55555556]
 [1.          0.66666667]]
Actual Output:
[[0.92]
 [0.86]
 [0.89]]
Predicted Output:

```

```

[[0.90736286]
[0.89239737]
[0.90632002]]
-----Epoch- 1 Ends-----

-----Epoch- 2 Starts-----
Input:
[[0.66666667 1.          ]
[0.33333333 0.55555556]
[1.          0.66666667]]
Actual Output:
[[0.92]
[0.86]
[0.89]]
Predicted Output:
[[0.90730647]
[0.89233865]
[0.90626389]]
-----Epoch- 2 Ends-----

-----Epoch- 3 Starts-----
Input:
[[0.66666667 1.          ]
[0.33333333 0.55555556]
[1.          0.66666667]]
Actual Output:
[[0.92]
[0.86]
[0.89]]
Predicted Output:
[[0.90725029]
[0.89228014]
[0.90620796]]
-----Epoch- 3 Ends-----

-----Epoch- 4 Starts-----
Input:
[[0.66666667 1.          ]
[0.33333333 0.55555556]
[1.          0.66666667]]
Actual Output:
[[0.92]
[0.86]
[0.89]]
Predicted Output:
[[0.90719432]
[0.89222186]
[0.90618833]]

```

Experiment 5

```

from math import sqrt
from statistics import mode

# -----
# TRAINING DATA
# -----

l = [
    [33.6, 50, 1],
    [26.6, 30, 0],
    [23.4, 40, 0],
    [43.1, 67, 0],
    [35.3, 23, 1],
    [35.9, 67, 1],
    [36.7, 45, 1],
    [25.7, 46, 0],
    [23.3, 29, 0],
    [31, 56, 1]
]

# -----
# TEST DATA
# -----

n = [43.6, 40]

# -----
# VALUE OF K
# -----

k = 3

```

```

# -----
# CALCULATE EUCLIDEAN DISTANCE
# -----

m = []

for i in l:

    a = 0

    for j in range(len(n)):

        a += (i[j] - n[j]) * (i[j] - n[j])

    m.append(sqrt(a))

# -----
# SORT DISTANCES
# -----

a = sorted(m)

# -----
# FIND K NEAREST NEIGHBOURS
# -----

x = []

for i in range(k):

    x.append(m.index(a[i]))

    # Avoid duplicate index problem
    m[m.index(a[i])] = float("inf")

# -----
# GET CLASS LABELS
# -----

y = []

print("The K nearest neighbours are:")

for i in x:

    print(l[i])

    y.append(l[i][-1])

# -----
# PREDICT CLASS
# -----

print()

print("Classes of nearest neighbours:", y)

print("Result -->", mode(y))

```

```

The K nearest neighbours are:
[36.7, 45, 1]
[33.6, 50, 1]
[25.7, 46, 0]

Classes of nearest neighbours: [1, 1, 0]
Result --> 1

```

Experiment 6

```

from math import sqrt
from statistics import mode

# -----
# TRAINING DATA
# -----

l = [

```

```

[33.6, 50, 1],
[26.6, 30, 0],
[23.4, 40, 0],
[43.1, 67, 0],
[35.3, 23, 1],
[35.9, 67, 1],
[36.7, 45, 1],
[25.7, 46, 0],
[23.3, 29, 0],
[31, 56, 1]
]

# -----
# TEST DATA
# -----

n = [43.6, 40]

# -----
# VALUE OF K
# -----

k = 3

# -----
# CALCULATE EUCLIDEAN DISTANCE
# -----

m = []

for i in l:

    a = 0

    for j in range(len(n)):

        a += (i[j] - n[j]) * (i[j] - n[j])

    m.append(sqrt(a))

# -----
# SORT DISTANCES
# -----

a = sorted(m)

# -----
# FIND K NEAREST NEIGHBOURS
# -----

x = []

for i in range(k):

    x.append(m.index(a[i]))

    # Avoid duplicate index problem
    m[m.index(a[i])] = float("inf")

# -----
# GET CLASS LABELS
# -----

y = []

print("The K nearest neighbours are:")

for i in x:

    print(l[i])

    y.append(l[i][-1])

# -----
# PREDICT CLASS

```

```
# -----

print()

print("Classes of nearest neighbours:", y)

print("Result -->", mode(y))
```

```
Iris Dataset:
      sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1           3.5           1.4           0.2
1                4.9           3.0           1.4           0.2
2                4.7           3.2           1.3           0.2
3                4.6           3.1           1.5           0.2
4                5.0           3.6           1.4           0.2
..                ...           ...           ...           ...
145              6.7           3.0           5.2           2.3
146              6.3           2.5           5.0           1.9
147              6.5           3.0           5.2           2.0
148              6.2           3.4           5.4           2.3
149              5.9           3.0           5.1           1.8
```

```
      species
0           0
1           0
2           0
3           0
4           0
..          ...
145         2
146         2
147         2
148         2
149         2
```

[150 rows x 5 columns]

Training samples: 105

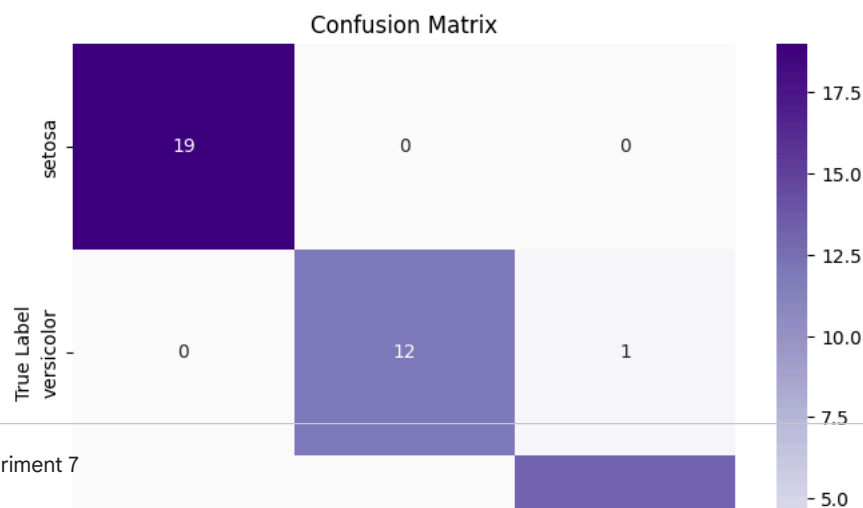
Testing samples: 45

Accuracy: 0.9777777777777777

Accuracy Percentage: 97.77777777777777 %

Confusion Matrix:

```
[[19  0  0]
 [ 0 12  1]
 [ 0  0 13]]
```



Experiment 7

```
# =====
# LINEAR REGRESSION
# =====

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# =====
# STEP 1: SET RANDOM SEED
# =====

np.random.seed(0)

# =====
# STEP 2: CREATE DATA
```



```
# =====

X = np.linspace(
    0,
    10,
    100
).reshape(-1, 1)

y = (
    2 * X
    + 1
    + np.random.randn(100, 1)
)

# =====
# STEP 3: CREATE LINEAR REGRESSION MODEL
# =====

lr_model = LinearRegression()

# =====
# STEP 4: TRAIN THE MODEL
# =====

lr_model.fit(
    X,
    y
)

# =====
# STEP 5: DISPLAY COEFFICIENT AND INTERCEPT
# =====

print(
    "Coefficients:",
    lr_model.coef_
)

print(
    "Intercept:",
    lr_model.intercept_
)

# =====
# STEP 6: MAKE PREDICTIONS
# =====

y_pred = lr_model.predict(X)

# =====
# STEP 7: PLOT DATA AND REGRESSION LINE
# =====

plt.scatter(
    X,
    y,
    color="blue",
    label="Actual Data"
)

plt.plot(
    X,
    y_pred,
    color="red",
    linewidth=3,
    label="Regression Line"
)

# =====
# STEP 8: ADD LABELS
# =====

plt.title(
    "Linear Regression"
)
```

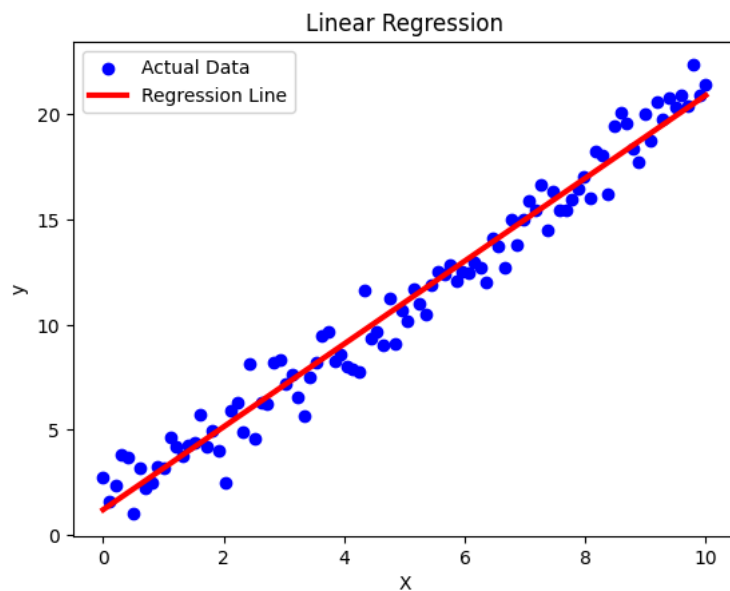
```
plt.xlabel(
    "x"
)

plt.ylabel(
    "y"
)

plt.legend()

plt.show()
```

Coefficients: $[[1.97026731]]$
 Intercept: $[1.20847145]$



Experiment 8

```
# =====
# LOGISTIC REGRESSION CLASSIFICATION
# =====

# Step 1: Import required modules

from sklearn.datasets import make_classification
from matplotlib import pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score
import pandas as pd

# =====
# Step 2: Generate the dataset
# =====

x, y = make_classification(
    n_samples=100,
    n_features=1,
    n_classes=2,
    n_clusters_per_class=1,
    flip_y=0.03,
    n_informative=1,
    n_redundant=0,
    n_repeated=0,
    random_state=42
)

print("Target values:")
print(y)

# =====
# Step 3: Visualize the dataset
# =====

plt.scatter(
    x,
```

```
        y,
        c=y,
        cmap="rainbow"
    )

plt.title(
    "Scatter Plot of Logistic Regression"
)

plt.xlabel("Feature")

plt.ylabel("Class")

plt.show()

# =====
# Step 4: Split the dataset
# =====

x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.25,
    random_state=1
)

print("\nTraining data shape:")
print(x_train.shape)

print("\nTesting data shape:")
print(x_test.shape)

# =====
# Step 5: Perform Logistic Regression
# =====

log_reg = LogisticRegression()

log_reg.fit(
    x_train,
    y_train
)

# =====
# Step 6: Make predictions
# =====

y_pred = log_reg.predict(
    x_test
)

print("\nPredicted values:")
print(y_pred)

# =====
# Step 7: Calculate accuracy
# =====

accuracy = accuracy_score(
    y_test,
    y_pred
)

print("\nAccuracy:")
print(accuracy)

print(
    "Accuracy Percentage:",
    accuracy * 100,
    "%"
)

# =====
# Step 8: Display Confusion Matrix
# =====

cm = confusion_matrix(
```

```
        y_test,
        y_pred
    )

    print("\nConfusion Matrix:")
    print(cm)

    # =====
    # Step 9: Plot Confusion Matrix
    # =====

    plt.figure(
        figsize=(6, 5)
    )

    plt.imshow(
        cm,
        interpolation="nearest",
        cmap="Blues"
    )

    plt.title(
        "Confusion Matrix"
    )

    plt.colorbar()

    plt.xlabel(
        "Predicted Label"
    )

    plt.ylabel(
        "True Label"
    )

    plt.xticks(
        [0, 1],
        ["Class 0", "Class 1"]
    )

    plt.yticks(
        [0, 1],
        ["Class 0", "Class 1"]
    )

    # Display values inside the matrix
    for i in range(2):

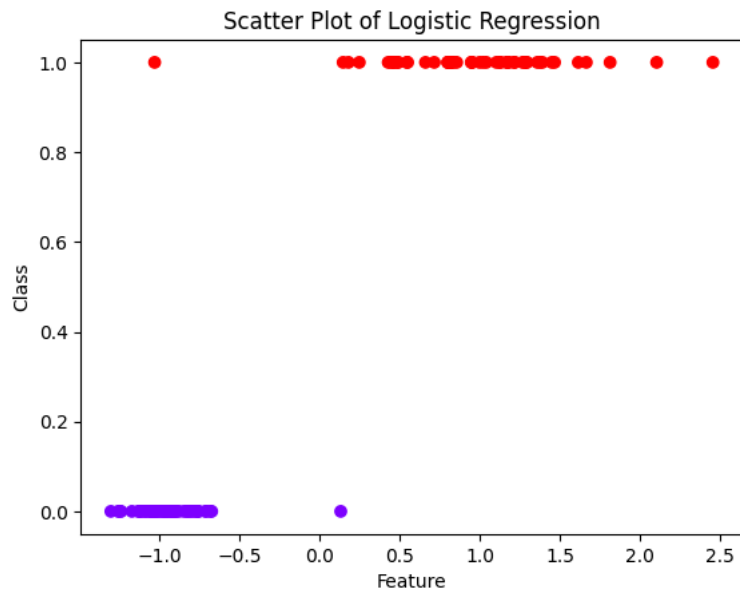
        for j in range(2):

            plt.text(
                j,
                i,
                cm[i, j],
                ha="center",
                va="center"
            )

    plt.show()
```

Target values:

```
[0 1 1 1 0 1 1 0 1 1 1 1 1 0 1 0 0 1 1 0 1 1 1 1 1 1 1 0 0 0 1 1 0 0 0
 1 0 0 0 0 0 0 0 0 0 0 1 1 0 1 1 0 0 0 1 1 0 0 1 1 1 0 0 0 1 0 0 1 1 1 1 1
 1 0 0 1 0 0 1 1 1 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 1 1 1]
```



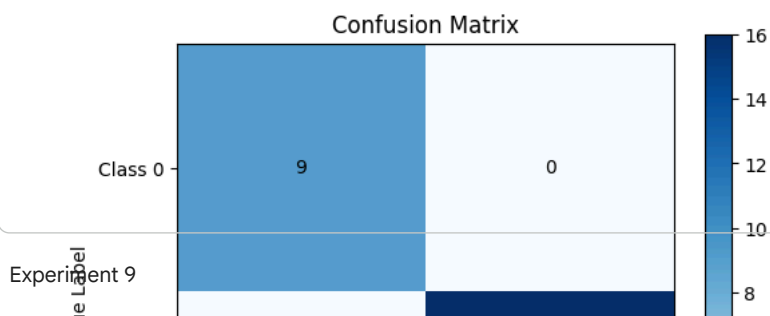
Training data shape:
(75, 1)

Testing data shape:
(25, 1)

Predicted values:
[1 0 1 1 0 0 0 1 1 0 1 0 1 1 1 1 0 0 0 1 1 1 1 1 1]

Accuracy:
1.0
Accuracy Percentage: 100.0 %

Confusion Matrix:
[[9 0]
[0 16]]



```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
import matplotlib.pyplot as plt
```

```
# =====
# SAMPLE DATA
# =====
```

```
X = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
```

```
y = np.array([2, 4, 5, 4, 5]).reshape(-1, 1)
```

```
# =====
# LINEAR REGRESSION
# =====
```

```
linear_model = LinearRegression()
```

```
linear_model.fit(X, y)
```

```
# Predict new value
X_new = np.array([6]).reshape(-1, 1)
```

```

linear_prediction = linear_model.predict(X_new)

print("=====")
print("      LINEAR REGRESSION")
print("=====")

print("Coefficient:", linear_model.coef_)

print("Intercept:", linear_model.intercept_)

print("Prediction for X = 6:", linear_prediction)

# =====
# POLYNOMIAL REGRESSION
# =====

poly = PolynomialFeatures(
    degree=2
)

X_poly = poly.fit_transform(X)

polynomial_model = LinearRegression()

polynomial_model.fit(
    X_poly,
    y
)

# Predict new value
X_new_poly = poly.transform(X_new)

polynomial_prediction = polynomial_model.predict(
    X_new_poly
)

print("\n=====")
print("      POLYNOMIAL REGRESSION")
print("=====")

print("Coefficients:", polynomial_model.coef_)

print("Intercept:", polynomial_model.intercept_)

print("Prediction for X = 6:", polynomial_prediction)

# =====
# DISPLAY COMPARISON
# =====

print("\n=====")
print("      COMPARISON")
print("=====")

print(
    "Linear Regression Prediction:",
    linear_prediction[0][0]
)

print(
    "Polynomial Regression Prediction:",
    polynomial_prediction[0][0]
)

# =====
# PLOTTING
# =====

# Create smooth values for polynomial curve
X_curve = np.linspace(
    1,
    5,
    100
).reshape(-1, 1)

```

```
X_curve_poly = poly.transform(
    X_curve
)

y_linear = linear_model.predict(
    X_curve
)

y_polynomial = polynomial_model.predict(
    X_curve_poly
)

# Plot actual data
plt.scatter(
    X,
    y,
    label="Actual Data"
)

# Linear regression line
plt.plot(
    X_curve,
    y_linear,
    label="Linear Regression"
)

# Polynomial regression curve
plt.plot(
    X_curve,
    y_polynomial,
    label="Polynomial Regression"
)

# Labels
plt.xlabel("X")

plt.ylabel("y")

plt.title(
    "Linear and Polynomial Regression"
)

plt.legend()

plt.show()
```

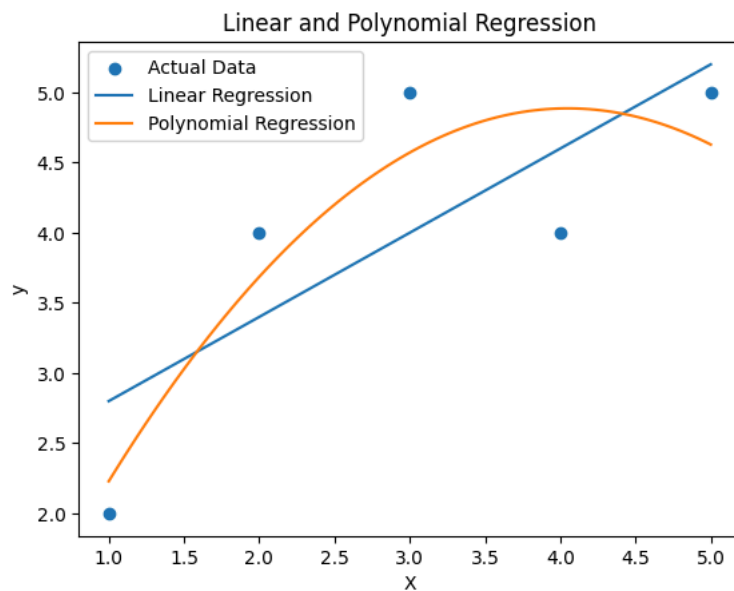
```

=====
LINEAR REGRESSION
=====
Coefficient: [[0.6]]
Intercept: [2.2]
Prediction for X = 6: [[5.8]]

=====
POLYNOMIAL REGRESSION
=====
Coefficients: [[ 0.          2.31428571 -0.28571429]]
Intercept: [0.2]
Prediction for X = 6: [[3.8]]

=====
COMPARISON
=====
Linear Regression Prediction: 5.8
Polynomial Regression Prediction: 3.7999999999999967

```



Experiment 10

```

import numpy as np
from scipy.stats import norm

# =====
# DATA
# =====

data = np.array([
    1.2,
    2.3,
    0.7,
    1.6,
    1.1,
    1.8,
    0.9,
    2.2
])

# =====
# INITIAL PARAMETERS
# =====

mu1 = 0
mu2 = 1

sigma1 = 1
sigma2 = 1

p1 = 0.5
p2 = 0.5

# =====
# EM ALGORITHM

```



```

# =====

for i in range(10):

    # -----
    # E-STEP
    # -----

    # Calculate likelihood of each data point
    # under Gaussian distribution 1 and 2

    likelihood1 = norm.pdf(
        data,
        mu1,
        sigma1
    )

    likelihood2 = norm.pdf(
        data,
        mu2,
        sigma2
    )

    # Calculate responsibility/weights

    denominator = (
        p1 * likelihood1
        +
        p2 * likelihood2
    )

    weight1 = (
        p1 * likelihood1
        /
        denominator
    )

    weight2 = (
        p2 * likelihood2
        /
        denominator
    )

    # -----
    # M-STEP
    # -----

    # Update mean

    mu1 = (
        np.sum(weight1 * data)
        /
        np.sum(weight1)
    )

    mu2 = (
        np.sum(weight2 * data)
        /
        np.sum(weight2)
    )

    # Update standard deviation

    sigma1 = np.sqrt(
        np.sum(
            weight1 * (data - mu1) ** 2
        )
        /
        np.sum(weight1)
    )

    sigma2 = np.sqrt(
        np.sum(
            weight2 * (data - mu2) ** 2
        )
        /
        np.sum(weight2)
    )

```

```

# Update mixing probabilities

p1 = np.mean(weight1)

p2 = np.mean(weight2)

# =====
# FINAL ESTIMATES
# =====

print("Final Estimates of Parameters")
print("=====")

print("mu1:", mu1)

print("mu2:", mu2)

print("sigma1:", sigma1)

print("sigma2:", sigma2)

print("p1:", p1)

print("p2:", p2)

```

```

Final Estimates of Parameters
=====
mu1: 0.9545902456963998
mu2: 1.7595212637782114
sigma1: 0.19986282179149245
sigma2: 0.47713642731204714
p1: 0.3534728534331289
p2: 0.6465271465668712

```

Experiment 11

```

# =====
# CREDIT SCORE PREDICTION USING RANDOM FOREST
# =====

import pandas as pd
import numpy as np
import os

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# =====
# STEP 1: CREATE SAMPLE DATASET IF FILE DOES NOT EXIST
# =====

file_name = "CREDITSCORE.csv"

if not os.path.exists(file_name):

    sample_data = {
        "Annual_Income": [
            50000, 60000, 45000, 80000, 35000,
            70000, 55000, 90000, 40000, 65000,
            75000, 48000, 85000, 52000, 62000
        ],
        "Monthly_Inhand_Salary": [
            4000, 5000, 3750, 6666, 2916,
            5833, 4583, 7500, 3333, 5416,
            6250, 4000, 7083, 4333, 5166
        ],
        "Num_Bank_Accounts": [
            3, 4, 2, 2, 6,
            3, 5, 1, 7, 4,
            2, 5, 2, 4, 3
        ],
        "Num_Credit_Card": [
            4, 5, 3, 2, 6,
            4, 5, 2, 7, 4,
            3, 6, 2, 5, 4
        ]
    }

```

```

    ],

    "Interest_Rate": [
        10, 12, 8, 7, 18,
        9, 14, 6, 20, 11,
        8, 16, 7, 13, 10
    ],

    "Num_of_Loan": [
        2, 3, 1, 1, 5,
        2, 4, 1, 6, 3,
        2, 5, 1, 4, 2
    ],

    "Delay_from_due_date": [
        5, 8, 3, 2, 18,
        6, 12, 1, 25, 7,
        4, 20, 2, 10, 5
    ],

    "Num_of_Delayed_Payment": [
        2, 3, 1, 1, 7,
        2, 5, 0, 9, 3,
        1, 8, 1, 4, 2
    ],

    "Credit_Mix": [
        3, 1, 3, 3, 0,
        3, 1, 3, 0, 1,
        3, 0, 3, 1, 3
    ],

    "Outstanding_Debt": [
        1000, 2500, 800, 500, 7000,
        1500, 4000, 300, 9000, 2800,
        1200, 6500, 600, 3500, 1800
    ],

    "Credit_History_Age": [
        120, 100, 140, 180, 50,
        130, 90, 200, 30, 110,
        160, 45, 190, 80, 125
    ],

    "Monthly_Balance": [
        3000, 3500, 2800, 5000, 1800,
        4000, 3200, 6000, 1500, 3700,
        4500, 2000, 5200, 2900, 3400
    ],

    "Credit_Score": [
        2, 1, 2, 2, 0,
        2, 1, 2, 0, 1,
        2, 0, 2, 1, 2
    ]
}

data = pd.DataFrame(sample_data)

data.to_csv(
    file_name,
    index=False
)

print("CREDITSCORE.csv was created successfully.")

# =====
# STEP 2: LOAD DATASET
# =====

data = pd.read_csv(file_name)

print("\nFirst 5 rows:")
print(data.head())

print("\nDataset Information:")
data.info()

# =====
# STEP 3: SELECT FEATURES

```

```

# =====

features = [
    "Annual_Income",
    "Monthly_Inhand_Salary",
    "Num_Bank_Accounts",
    "Num_Credit_Card",
    "Interest_Rate",
    "Num_of_Loan",
    "Delay_from_due_date",
    "Num_of_Delayed_Payment",
    "Credit_Mix",
    "Outstanding_Debt",
    "Credit_History_Age",
    "Monthly_Balance"
]

# =====
# STEP 4: CREATE X AND Y
# =====

X = data[features]

y = data["Credit_Score"]

# =====
# STEP 5: SPLIT DATA
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.33,
    random_state=42
)

# =====
# STEP 6: RANDOM FOREST CLASSIFIER
# =====

model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

# =====
# STEP 7: TRAIN MODEL
# =====

model.fit(
    X_train,
    y_train
)

# =====
# STEP 8: TEST MODEL
# =====

y_pred = model.predict(X_test)

accuracy = accuracy_score(
    y_test,
    y_pred
)

print("\n=====")
print("MODEL ACCURACY")
print("=====")

print(
    "Accuracy:",
    accuracy
)

print(
    "Accuracy Percentage:",
    accuracy * 100,

```

```

        "%"
    )

# =====
# STEP 9: USER INPUT
# =====

print("\n=====")
print("CREDIT SCORE PREDICTION")
print("=====")

a = float(
    input("Annual Income: ")
)

b = float(
    input("Monthly Inhand Salary: ")
)

c = float(
    input("Number of Bank Accounts: ")
)

d = float(
    input("Number of Credit Cards: ")
)

e = float(
    input("Interest Rate: ")
)

f = float(
    input("Number of Loans: ")
)

g = float(
    input("Average number of days delayed by the person: ")
)

h = float(
    input("Number of delayed payments: ")
)

print("\nCredit Mix:")
print("0 = Bad")
print("1 = Standard")
print("3 = Good")

i = float(
    input("Credit Mix: ")
)

j = float(
    input("Outstanding Debt: ")
)

k = float(
    input("Credit History Age (months): ")
)

l = float(
    input("Monthly Balance: ")
)

# =====
# STEP 10: CREATE INPUT DATA
# =====

new_data = pd.DataFrame(
    [[
        a, b, c, d, e, f,
        g, h, i, j, k, l
    ]],
    columns=features
)

# =====

```

```
# STEP 11: PREDICT CREDIT SCORE
# =====

prediction = model.predict(
    new_data
)

# =====
# STEP 12: DISPLAY RESULT
# =====

print("\n=====")

print(
    "Predicted Credit Score =",
    prediction[0]
)

print("=====")
```

CREDITSCORE.csv was created successfully.

First 5 rows:

	Annual_Income	Monthly_Inhand_Salary	Num_Bank_Accounts	Num_Credit_Card	\
0	50000	4000	3	4	
1	60000	5000	4	5	
2	45000	3750	2	3	
3	80000	6666	2	2	
4	35000	2916	6	6	

	Interest_Rate	Num_of_Loan	Delay_from_due_date	Num_of_Delayed_Payment	\
0	10	2	5	2	
1	12	3	8	3	
2	8	1	3	1	
3	7	1	2	1	
4	18	5	18	7	

	Credit_Mix	Outstanding_Debt	Credit_History_Age	Monthly_Balance	\
0	3	1000	120	3000	
1	1	2500	100	3500	
2	3	800	140	2800	
3	3	500	180	5000	
4	0	7000	50	1800	

Credit_Score

0	2
1	1
2	2
3	2
4	0

Dataset Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 15 entries, 0 to 14

Data columns (total 13 columns):

#	Column	Non-Null Count	Dtype
0	Annual_Income	15 non-null	int64
1	Monthly_Inhand_Salary	15 non-null	int64
2	Num_Bank_Accounts	15 non-null	int64
3	Num_Credit_Card	15 non-null	int64
4	Interest_Rate	15 non-null	int64
5	Num_of_Loan	15 non-null	int64
6	Delay_from_due_date	15 non-null	int64
7	Num_of_Delayed_Payment	15 non-null	int64
8	Credit_Mix	15 non-null	int64
9	Outstanding_Debt	15 non-null	int64
10	Credit_History_Age	15 non-null	int64
11	Monthly_Balance	15 non-null	int64
12	Credit_Score	15 non-null	int64

dtypes: int64(13)

memory usage: 1.7 KB

```
=====
MODEL ACCURACY
=====
Accuracy: 1.0
Accuracy Percentage: 100.0 %
```

Experiment 12

```
# =====
# 12. IRIS FLOWER CLASSIFICATION USING KNN
# =====
```

```
import os
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# =====
# STEP 1: CREATE IRIS.CSV IF IT DOES NOT EXIST
# =====

file_name = "IRIS.csv"

if not os.path.exists(file_name):

    from sklearn.datasets import load_iris

    iris_data = load_iris()

    iris = pd.DataFrame(
        iris_data.data,
        columns=[
            "sepal_length",
            "sepal_width",
            "petal_length",
            "petal_width"
        ]
    )

    # Convert target numbers to species names
    iris["species"] = [
        iris_data.target_names[i]
        for i in iris_data.target
    ]

    iris.to_csv(
        file_name,
        index=False
    )

    print("IRIS.csv created successfully.")

# =====
# STEP 2: LOAD IRIS DATASET
# =====

iris = pd.read_csv(file_name)

# =====
# STEP 3: DISPLAY DATASET
# =====

print("\nFirst 5 rows:")
print(iris.head())

print("\nDataset Description:")
print(iris.describe())

print("\nTarget Labels:")
print(iris["species"].unique())

# =====
# STEP 4: VISUALIZE DATA
# =====

import plotly.express as px

fig = px.scatter(
    iris,
    x="sepal_width",
    y="sepal_length",
    color="species",
    title="Iris Flower Classification"
)

fig.show()

# =====
# STEP 5: SEPARATE FEATURES AND TARGET
```

```

# =====

x = iris.drop(
    "species",
    axis=1
)

y = iris["species"]

# =====
# STEP 6: SPLIT DATA INTO TRAINING AND TESTING
# =====

from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.2,
    random_state=0
)

print("\nTraining data shape:")
print(x_train.shape)

print("\nTesting data shape:")
print(x_test.shape)

# =====
# STEP 7: CREATE KNN CLASSIFIER
# =====

from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(
    n_neighbors=1
)

# =====
# STEP 8: TRAIN KNN MODEL
# =====

knn.fit(
    x_train,
    y_train
)

# =====
# STEP 9: TEST DATA
# =====

x_new = np.array([
    [6, 2.9, 1, 0.2]
])

# =====
# STEP 10: MAKE PREDICTION
# =====

prediction = knn.predict(
    x_new
)

# =====
# STEP 11: DISPLAY PREDICTION
# =====

print("\n=====")
print("      IRIS FLOWER PREDICTION")
print("=====")

print(
    "Input:",
    x_new
)

```



```
print(
    "Prediction:",
    prediction
)

print("=====")
```

IRIS.csv created successfully.

First 5 rows:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

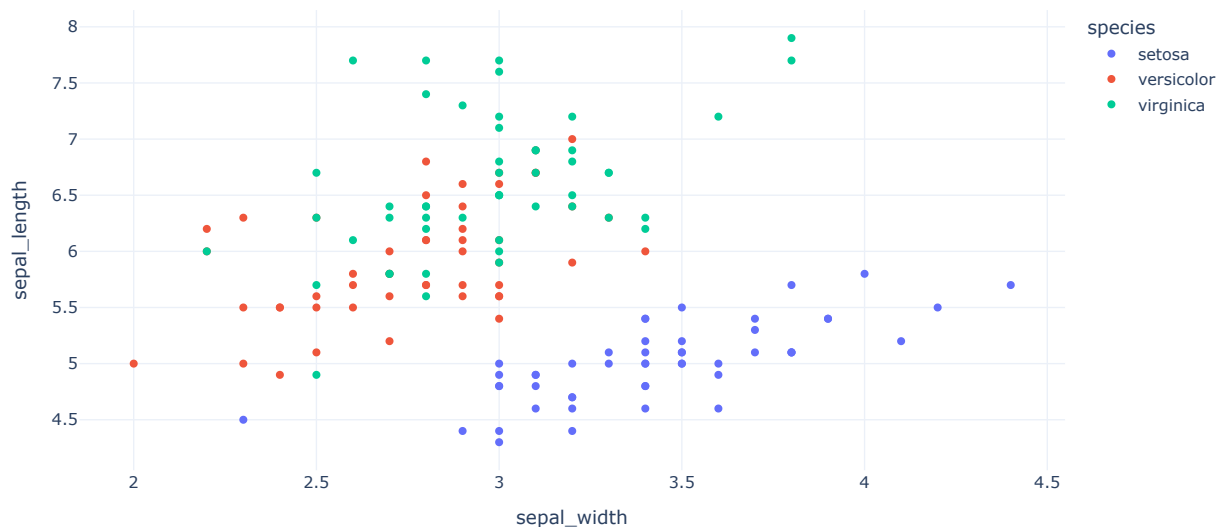
Dataset Description:

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

Target Labels:

['setosa' 'versicolor' 'virginica']

Iris Flower Classification



Training data shape:
(120, 4)

Testing data shape:
(30, 4)

Experiment 13

```
=====
# =====
# CAR PRICE PREDICTION USING DECISION TREE REGRESSION
# =====

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# =====
# STEP 1: CREATE CarPrice.csv AUTOMATICALLY
# =====
```

```

file_name = "CarPrice.csv"

if not os.path.exists(file_name):

    car_data = {
        "CarName": [
            "car1", "car2", "car3", "car4", "car5",
            "car6", "car7", "car8", "car9", "car10",
            "car11", "car12", "car13", "car14", "car15",
            "car16", "car17", "car18", "car19", "car20"
        ],

        "symboling": [
            3, 3, 1, 2, 2,
            1, 1, 2, 3, 2,
            0, 1, 2, 0, 1,
            2, 3, 0, 1, 2
        ],

        "wheelbase": [
            88.6, 88.6, 94.5, 99.8, 99.4,
            99.8, 105.8, 105.8, 88.6, 93.7,
            96.5, 96.5, 102.4, 102.4, 102.4,
            104.3, 104.3, 107.9, 107.9, 109.1
        ],

        "carlength": [
            168.8, 168.8, 171.2, 176.6, 176.6,
            177.3, 192.7, 192.7, 168.8, 157.3,
            167.3, 167.3, 175.6, 175.6, 175.6,
            183.1, 183.1, 186.7, 186.7, 189.0
        ],

        "carwidth": [
            64.1, 64.1, 65.5, 66.2, 66.4,
            66.3, 68.4, 68.4, 64.1, 63.8,
            65.4, 65.4, 66.5, 66.5, 66.5,
            67.9, 67.9, 68.4, 68.4, 68.9
        ],

        "carheight": [
            48.8, 48.8, 52.4, 54.3, 54.3,
            54.5, 56.7, 56.7, 48.8, 50.8,
            53.0, 53.0, 54.9, 54.9, 54.9,
            55.7, 55.7, 56.2, 56.2, 56.8
        ],

        "curbweight": [
            2548, 2548, 2823, 2337, 2824,
            2500, 2954, 2954, 2500, 2261,
            2395, 2395, 2500, 2500, 2500,
            2600, 2600, 2800, 2800, 3000
        ],

        "enginesize": [
            130, 130, 152, 109, 136,
            136, 164, 164, 109, 108,
            110, 110, 140, 140, 140,
            120, 120, 150, 150, 160
        ],

        "bore ratio": [
            3.47, 3.47, 2.68, 3.19, 3.19,
            3.19, 3.31, 3.31, 3.19, 3.62,
            3.62, 3.62, 3.78, 3.78, 3.78,
            3.54, 3.54, 3.78, 3.78, 3.80
        ],

        "stroke": [
            2.68, 2.68, 3.47, 3.40, 3.40,
            3.40, 3.46, 3.46, 3.40, 3.39,
            3.39, 3.39, 3.12, 3.12, 3.12,
            3.07, 3.07, 3.15, 3.15, 3.20
        ],

        "compressionratio": [
            9.0, 9.0, 9.0, 10.0, 8.5,
            8.5, 8.0, 8.0, 8.5, 8.8,
            8.8, 8.8, 9.5, 9.5, 9.5,
            10.0, 10.0, 9.8, 9.8, 9.5
        ],
    }

```

```

        "horsepower": [
            111, 111, 154, 102, 115,
            110, 121, 121, 102, 70,
            70, 70, 90, 90, 90,
            110, 110, 130, 130, 150
        ],

        "peakrpm": [
            5000, 5000, 5000, 5500, 5500,
            5500, 5500, 5500, 5500, 4800,
            4800, 4800, 5200, 5200, 5200,
            5000, 5000, 5000, 5000, 5000
        ],

        "citympg": [
            21, 21, 19, 24, 18,
            20, 19, 19, 24, 38,
            38, 38, 30, 30, 30,
            28, 28, 25, 25, 22
        ],

        "highwaympg": [
            27, 27, 26, 30, 22,
            25, 25, 25, 30, 42,
            42, 42, 38, 38, 38,
            34, 34, 32, 32, 29
        ],

        "price": [
            13495, 16500, 16500, 13950, 17450,
            15250, 24500, 25500, 13800, 8058,
            8238, 8558, 12945, 13499, 13999,
            15500, 16500, 18900, 19500, 22000
        ]
    }

}

# Create DataFrame
car_df = pd.DataFrame(car_data)

# Save as CSV
car_df.to_csv(
    file_name,
    index=False
)

print("CarPrice.csv created successfully!")

# =====
# STEP 2: READ CSV FILE
# =====

data = pd.read_csv(
    file_name
)

# =====
# STEP 3: DISPLAY DATA
# =====

print("\nFirst 5 rows:")
print(data.head())

print("\nDataset Shape:")
print(data.shape)

print("\nMissing Values:")
print(data.isnull().sum())

print("\nDataset Description:")
print(data.describe())

print("\nCar Names:")
print(data["CarName"].unique())

```

```
# =====
# STEP 4: CORRELATION
# =====

correlations = data.select_dtypes(
    include=np.number
).corr()

print("\nCorrelation Matrix:")
print(correlations)

# =====
# STEP 5: HEATMAP
# =====

plt.figure(
    figsize=(15, 10)
)

sns.heatmap(
    correlations,
    annot=True,
    cmap="coolwarm"
)

plt.title(
    "Car Price Correlation Heatmap"
)

plt.show()

# =====
# STEP 6: SELECT FEATURES
# =====

predict = "price"

selected_columns = [
    "symboling",
    "wheelbase",
    "carlength",
    "carwidth",
    "carheight",
    "curbweight",
    "enginesize",
    "boreratio",
    "stroke",
    "compressionratio",
    "horsepower",
    "peakrpm",
    "citympg",
    "highwaympg",
    "price"
]

data = data[
    selected_columns
]

# =====
# STEP 7: CREATE X AND Y
# =====

x = np.array(
    data.drop(
        [predict],
        axis=1
    )
)

y = np.array(
    data[predict]
)

# =====
# STEP 8: SPLIT DATA
```

```
# =====

xtrain, xtest, ytrain, ytest = train_test_split(
    x,
    y,
    test_size=0.2,
    random_state=42
)

print("\nTraining data:")
print(xtrain.shape)

print("\nTesting data:")
print(xtest.shape)

# =====
# STEP 9: CREATE DECISION TREE MODEL
# =====

model = DecisionTreeRegressor(
    random_state=42
)

# =====
# STEP 10: TRAIN MODEL
# =====

model.fit(
    xtrain,
    ytrain
)

# =====
# STEP 11: PREDICT
# =====

predictions = model.predict(
    xtest
)

# =====
# STEP 12: DISPLAY PREDICTIONS
# =====

print("\n=====")
print("CAR PRICE PREDICTION")
print("=====")

print("\nActual Prices:")
print(ytest)

print("\nPredicted Prices:")
print(predictions)

# =====
# STEP 13: MODEL EVALUATION
# =====

r2 = model.score(
    xtest,
    ytest
)

mae = mean_absolute_error(
    ytest,
    predictions
)

mse = mean_squared_error(
    ytest,
    predictions
)

rmse = np.sqrt(mse)
```

```
# =====
# STEP 14: DISPLAY RESULTS
# =====

print("\n=====")
print("MODEL RESULTS")
print("=====")

print("R² Score:", r2)

print("Mean Absolute Error:", mae)

print("Mean Squared Error:", mse)

print("Root Mean Squared Error:", rmse)

print("=====")
```

[Show hidden output](#)

Experiment 14

```
# =====
# HOUSE PRICE PREDICTION
# USING SVR AND LINEAR REGRESSION
# =====

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# =====
# STEP 1: CHECK CSV FILE
# =====

file_name = "HousePricePrediction.csv"

if not os.path.exists(file_name):

    print("HousePricePrediction.csv not found.")
    print("Creating sample dataset...")

    house_data = {
        "Id": range(1, 21),

        "MSZoning": [
            "RL", "RL", "RM", "RL", "RL",
            "RM", "RL", "RL", "RM", "RL",
            "RL", "RM", "RL", "RL", "RM",
            "RL", "RL", "RM", "RL", "RL"
        ],

        "LotArea": [
            8450, 9600, 11250, 9550, 14260,
            14115, 10084, 10382, 6120, 7420,
            11200, 11924, 12968, 10652, 10920,
            6120, 11622, 11241, 12925, 14375
        ],

        "OverallQual": [
            7, 6, 7, 7, 8,
            5, 8, 7, 6, 5,
            7, 6, 8, 7, 6,
            7, 7, 5, 6, 8
        ],

        "OverallCond": [
            5, 8, 5, 5, 5,
            5, 5, 6, 5, 6,
            5, 5, 5, 5, 5,
            7, 5, 6, 5, 5
        ],

        "YearBuilt": [
            2003, 1976, 2001, 1915, 2000,
            1993, 2004, 1973, 1931, 1939,
            1965, 2005, 1962, 2006, 1960,
            2001, 2004, 1998, 2005, 2006
```

```

    ],

    "GrLivArea": [
        1710, 1262, 1786, 1717, 2198,
        1362, 1694, 2090, 1774, 1077,
        1040, 2324, 912, 1494, 1253,
        854, 1004, 1296, 1114, 1339
    ],

    "BedroomAbvGr": [
        3, 3, 3, 3, 4,
        1, 3, 3, 2, 2,
        3, 4, 2, 3, 2,
        2, 2, 3, 3, 3
    ],

    "FullBath": [
        2, 2, 2, 1, 2,
        1, 2, 2, 2, 1,
        1, 3, 2, 2, 1,
        1, 1, 2, 2, 2
    ],

    "GarageCars": [
        2, 2, 2, 3, 3,
        2, 2, 2, 2, 1,
        1, 3, 2, 3, 1,
        2, 2, 2, 2, 2
    ],

    "SalePrice": [
        208500, 181500, 223500, 140000, 250000,
        143000, 307000, 200000, 129900, 118000,
        129500, 345000, 144000, 279500, 157000,
        132000, 149000, 90000, 159000, 189000
    ]
}

house_df = pd.DataFrame(house_data)

house_df.to_csv(
    file_name,
    index=False
)

print("HousePricePrediction.csv created successfully.")

# =====
# STEP 2: IMPORT DATASET
# =====

dataset = pd.read_csv(
    file_name
)

# =====
# STEP 3: EXPLORE DATASET
# =====

print("\n=====")
print("FIRST 5 ROWS")
print("=====")

print(
    dataset.head(5)
)

print("\nDataset Shape:")

print(
    dataset.shape
)

# =====
# STEP 4: IDENTIFY DATA TYPES
# =====

obj = (

```

```

        dataset.dtypes == "object"
    )

    object_cols = list(
        obj[obj].index
    )

    print(
        "\nCategorical variables:",
        len(object_cols)
    )

    # Integer columns
    int_ = (
        dataset.dtypes == "int64"
    )

    num_cols = list(
        int_[int_].index
    )

    print(
        "Integer variables:",
        len(num_cols)
    )

    # Float columns
    fl = (
        dataset.dtypes == "float64"
    )

    fl_cols = list(
        fl[fl].index
    )

    print(
        "Float variables:",
        len(fl_cols)
    )

    # =====
    # STEP 5: CORRELATION HEATMAP
    # =====

    plt.figure(
        figsize=(12, 6)
    )

    correlation = dataset.select_dtypes(
        include=np.number
    ).corr()

    sns.heatmap(
        correlation,
        cmap="BrBG",
        fmt=".2f",
        linewidths=2,
        annot=True
    )

    plt.title(
        "Correlation Heatmap"
    )

    plt.show()

    # =====
    # STEP 6: UNIQUE VALUES OF CATEGORICAL FEATURES
    # =====

    unique_values = []

    for col in object_cols:
        unique_values.append(
            dataset[col].unique().size
        )

```



```

plt.figure(
    figsize=(10, 6)
)

plt.title(
    "Number of Unique Values of Categorical Features"
)

plt.xticks(
    rotation=90
)

sns.barplot(
    x=object_cols,
    y=unique_values
)

plt.show()

# =====
# STEP 7: CATEGORICAL FEATURE DISTRIBUTION
# =====

if len(object_cols) > 0:

    plt.figure(
        figsize=(18, 10)
    )

    index = 1

    for col in object_cols:

        y = dataset[col].value_counts()

        plt.subplot(
            2,
            2,
            index
        )

        plt.xticks(
            rotation=90
        )

        sns.barplot(
            x=list(y.index),
            y=list(y.values)
        )

        plt.title(
            col
        )

        index += 1

    plt.tight_layout()

    plt.show()

# =====
# STEP 8: REMOVE ID COLUMN
# =====

if "Id" in dataset.columns:

    dataset.drop(
        ["Id"],
        axis=1,
        inplace=True
    )

# =====
# STEP 9: HANDLE MISSING VALUES
# =====

if "SalePrice" in dataset.columns:

```

```

dataset["SalePrice"] = dataset[
    "SalePrice"
].fillna(
    dataset["SalePrice"].mean()
)

# Remove remaining missing values
new_dataset = dataset.dropna()

print("\nMissing values after cleaning:")

print(
    new_dataset.isnull().sum()
)

# =====
# STEP 10: ONE-HOT ENCODING
# =====

from sklearn.preprocessing import OneHotEncoder

s = (
    new_dataset.dtypes == "object"
)

object_cols = list(
    s[s].index
)

print(
    "\nCategorical variables:"
)

print(
    object_cols
)

print(
    "Number of categorical features:",
    len(object_cols)
)

# =====
# STEP 11: ENCODE CATEGORICAL VARIABLES
# =====

if len(object_cols) > 0:

    # Compatibility with newer sklearn
    try:

        OH_encoder = OneHotEncoder(
            sparse_output=False,
            handle_unknown="ignore"
        )

    except TypeError:

        # For older sklearn versions
        OH_encoder = OneHotEncoder(
            sparse=False,
            handle_unknown="ignore"
        )

    OH_cols = pd.DataFrame(
        OH_encoder.fit_transform(
            new_dataset[object_cols]
        )
    )

    OH_cols.index = new_dataset.index

# New sklearn function

```

```

try:

    OH_cols.columns = (
        OH_encoder.get_feature_names_out(
            object_cols
        )
    )

except AttributeError:

    OH_cols.columns = (
        OH_encoder.get_feature_names(
            object_cols
        )
    )

# Remove original categorical columns
df_final = new_dataset.drop(
    object_cols,
    axis=1
)

# Combine numerical + encoded categorical columns
df_final = pd.concat(
    [df_final, OH_cols],
    axis=1
)

else:

    df_final = new_dataset.copy()

# =====
# STEP 12: CREATE X AND Y
# =====

X = df_final.drop(
    ["SalePrice"],
    axis=1
)

Y = df_final["SalePrice"]

# =====
# STEP 13: TRAIN TEST SPLIT
# =====

X_train, X_valid, Y_train, Y_valid = train_test_split(
    X,
    Y,
    train_size=0.8,
    test_size=0.2,
    random_state=0
)

print("\nTraining data shape:")
print(X_train.shape)

print("\nValidation data shape:")
print(X_valid.shape)

# =====
# STEP 14: SUPPORT VECTOR REGRESSION
# =====

from sklearn import svm

model_SVR = svm.SVR()

model_SVR.fit(
    X_train,
    Y_train
)

# Prediction

```

```

Y_pred_SVR = model_SVR.predict(
    X_valid
)

# =====
# STEP 15: SVR ERROR
# =====

from sklearn.metrics import mean_absolute_percentage_error

SVR_error = mean_absolute_percentage_error(
    Y_valid,
    Y_pred_SVR
)

print("\n=====")
print("SUPPORT VECTOR REGRESSION")
print("=====")

print(
    "Mean Absolute Percentage Error:",
    SVR_error
)

print(
    "MAPE Percentage:",
    SVR_error * 100,
    "%"
)

# =====
# STEP 16: LINEAR REGRESSION
# =====

from sklearn.linear_model import LinearRegression

model_LR = LinearRegression()

model_LR.fit(
    X_train,
    Y_train
)

# Prediction
Y_pred_LR = model_LR.predict(
    X_valid
)

# =====
# STEP 17: LINEAR REGRESSION ERROR
# =====

LR_error = mean_absolute_percentage_error(
    Y_valid,
    Y_pred_LR
)

print("\n=====")
print("LINEAR REGRESSION")
print("=====")

print(
    "Mean Absolute Percentage Error:",
    LR_error
)

print(
    "MAPE Percentage:",
    LR_error * 100,
    "%"
)

# =====
# STEP 18: COMPARISON

```

```
# =====

print("\n=====")
print("MODEL COMPARISON")
print("=====")

print(
    "SVR MAPE:",
    SVR_error * 100,
    "%"
)

print(
    "Linear Regression MAPE:",
    LR_error * 100,
    "%"
)

if SVR_error < LR_error:

    print(
        "\nBetter Model: Support Vector Regression"
    )

else:

    print(
        "\nBetter Model: Linear Regression"
    )

print("=====")
```

HousePricePrediction.csv not found.
 Creating sample dataset...
 HousePricePrediction.csv created successfully.

=====

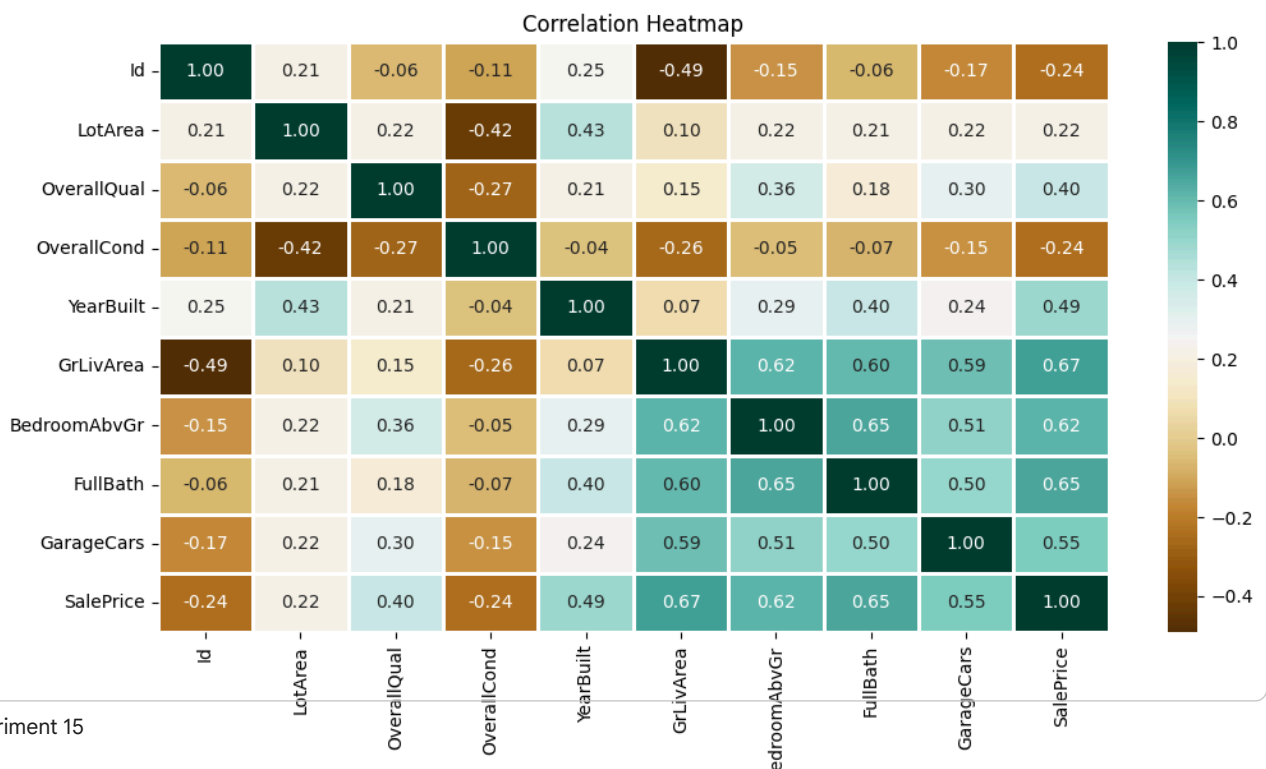
FIRST 5 ROWS

```
=====
      Id MSZoning LotArea OverallQual OverallCond YearBuilt GrLivArea \
0      1      RL   8450         7           5     2003      1710
1      2      RL   9600         6           8     1976      1262
2      3      RM  11250         7           5     2001      1786
3      4      RL   9550         7           5     1915      1717
4      5      RL  14260         8           5     2000      2198
```

```
      BedroomAbvGr FullBath GarageCars SalePrice
0                3         2           2    208500
1                3         2           2    181500
2                3         2           2    223500
3                3         1           3    140000
4                4         2           3    250000
```

Dataset Shape:
 (20, 11)

Categorical variables: 1
 Integer variables: 10
 Float variables: 0



Experiment 15

```
# =====
# GAUSSIAN NAIVE BAYES AND MULTINOMIAL NAIVE BAYES
# USING IRIS DATASET
# =====

from sklearn.naive_bayes import GaussianNB
from sklearn.naive_bayes import MultinomialNB
from sklearn import datasets
from sklearn.metrics import confusion_matrix, accuracy_score

# =====
# STEP 1: LOAD IRIS DATASET
# =====

iris = datasets.load_iris()

X = iris.data
y = iris.target

print("Iris Dataset:")
print(X)

print("\nTarget:")
print(y)
```

```

# =====
# STEP 2: CREATE GNB AND MNB MODELS
# =====

gnb = GaussianNB()

mnb = MultinomialNB()

# =====
# STEP 3: GAUSSIAN NAIVE BAYES
# =====

y_pred_gnb = gnb.fit(
    X,
    y
).predict(X)

# Confusion Matrix
cnf_matrix_gnb = confusion_matrix(
    y,
    y_pred_gnb
)

# Accuracy
accuracy_gnb = accuracy_score(
    y,
    y_pred_gnb
)

print("\n=====")
print("GAUSSIAN NAIVE BAYES")
print("=====")

print(
    "Confusion Matrix of GNB:"
)

print(
    cnf_matrix_gnb
)

print(
    "Accuracy of GNB:",
    accuracy_gnb
)

# =====
# STEP 4: MULTINOMIAL NAIVE BAYES
# =====

y_pred_mnb = mnb.fit(
    X,
    y
).predict(X)

# Confusion Matrix
cnf_matrix_mnb = confusion_matrix(
    y,
    y_pred_mnb
)

# Accuracy
accuracy_mnb = accuracy_score(
    y,
    y_pred_mnb
)

print("\n=====")
print("MULTINOMIAL NAIVE BAYES")
print("=====")

print(

```

```

    "Confusion Matrix of MNB:"
)

print(
    cnf_matrix_mnb
)

print(
    "Accuracy of MNB:",
    accuracy_mnb
)

# =====
# STEP 5: FINAL COMPARISON
# =====

print("\n=====")
print("FINAL COMPARISON")
print("=====")

print(
    "Gaussian NB Accuracy:",
    accuracy_gnb
)

print(
    "Multinomial NB Accuracy:",
    accuracy_mnb
)

```

```

Iris Dataset:
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]
 [5.4 3.9 1.7 0.4]
 [4.6 3.4 1.4 0.3]
 [5.  3.4 1.5 0.2]
 [4.4 2.9 1.4 0.2]
 [4.9 3.1 1.5 0.1]
 [5.4 3.7 1.5 0.2]
 [4.8 3.4 1.6 0.2]
 [4.8 3.  1.4 0.1]
 [4.3 3.  1.1 0.1]
 [5.8 4.  1.2 0.2]
 [5.7 4.4 1.5 0.4]
 [5.4 3.9 1.3 0.4]
 [5.1 3.5 1.4 0.3]
 [5.7 3.8 1.7 0.3]
 [5.1 3.8 1.5 0.3]
 [5.4 3.4 1.7 0.2]
 [5.1 3.7 1.5 0.4]
 [4.6 3.6 1.  0.2]
 [5.1 3.3 1.7 0.5]
 [4.8 3.4 1.9 0.2]
 [5.  3.  1.6 0.2]
 [5.  3.4 1.6 0.4]
 [5.2 3.5 1.5 0.2]
 [5.2 3.4 1.4 0.2]
 [4.7 3.2 1.6 0.2]
 [4.8 3.1 1.6 0.2]
 [5.4 3.4 1.5 0.4]
 [5.2 4.1 1.5 0.1]
 [5.5 4.2 1.4 0.2]
 [4.9 3.1 1.5 0.2]
 [5.  3.2 1.2 0.2]
 [5.5 3.5 1.3 0.2]
 [4.9 3.6 1.4 0.1]
 [4.4 3.  1.3 0.2]
 [5.1 3.4 1.5 0.2]
 [5.  3.5 1.3 0.3]
 [4.5 2.3 1.3 0.3]
 [4.4 3.2 1.3 0.2]
 [5.  3.5 1.6 0.6]
 [5.1 3.8 1.9 0.4]
 [4.8 3.  1.4 0.3]
 [5.1 3.8 1.6 0.2]
 [4.6 3.2 1.4 0.2]
 [5.3 3.7 1.5 0.2]
 [5.  3.3 1.4 0.2]
 [7.  3.2 4.7 1.4]
 [6.4 3.2 4.5 1.5]
 [6.9 3.1 4.9 1.5]
 [5.5 2.3 4.  1.3]
 [6.5 2.8 4.6 1.5]

```



```
[5.7 2.8 4.5 1.3]
[6.2 2.4 4.7 1.6]
```

Experiment 16

```
# =====
# COMPARISON OF CLASSIFICATION ALGORITHMS
# USING IRIS DATASET
# =====

import pandas as pd
import numpy as np

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import BernoulliNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import PassiveAggressiveClassifier

from sklearn.metrics import classification_report

# =====
# STEP 1: LOAD IRIS DATASET
# =====

iris_data = load_iris()

# =====
# STEP 2: CREATE DATAFRAME
# =====

iris = pd.DataFrame(
    iris_data.data,
    columns=[
        "sepal_length",
        "sepal_width",
        "petal_length",
        "petal_width"
    ]
)

# Add species column
iris["species"] = [
    iris_data.target_names[i]
    for i in iris_data.target
]

# =====
# STEP 3: DISPLAY DATASET
# =====

print("First 5 rows of Iris Dataset:")
print(iris.head())

print("\nDataset Shape:")
print(iris.shape)

# =====
# STEP 4: SEPARATE FEATURES AND TARGET
# =====

x = iris.drop(
    "species",
    axis=1
)

y = iris["species"]

# =====
# STEP 5: TRAIN TEST SPLIT
# =====
```

```
x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.2,
    random_state=42
)

print("\nTraining Data:")
print(x_train.shape)

print("\nTesting Data:")
print(x_test.shape)

# =====
# STEP 6: CREATE CLASSIFICATION MODELS
# =====

decisiontree = DecisionTreeClassifier(
    random_state=42
)

logisticregression = LogisticRegression(
    max_iter=200
)

knearestclassifier = KNeighborsClassifier()

bernoulli_naiveBayes = BernoulliNB()

passiveAggressive = PassiveAggressiveClassifier(
    random_state=42
)

# =====
# STEP 7: TRAIN MODELS
# =====

knearestclassifier.fit(
    x_train,
    y_train
)

decisiontree.fit(
    x_train,
    y_train
)

logisticregression.fit(
    x_train,
    y_train
)

bernoulli_naiveBayes.fit(
    x_train,
    y_train
)

passiveAggressive.fit(
    x_train,
    y_train
)

# =====
# STEP 8: CALCULATE ACCURACY
# =====

knn_score = knearestclassifier.score(
    x_test,
    y_test
)

dt_score = decisiontree.score(
    x_test,
    y_test
)

lr_score = logisticregression.score(
```

```

        x_test,
        y_test
    )

    bnb_score = bernoulli_naiveBayes.score(
        x_test,
        y_test
    )

    pa_score = passiveAggressive.score(
        x_test,
        y_test
    )

# =====
# STEP 9: CREATE SCORE TABLE
# =====

data1 = {
    "Classification Algorithms": [
        "KNN Classifier",
        "Decision Tree Classifier",
        "Logistic Regression",
        "Bernoulli Naive Bayes",
        "Passive Aggressive Classifier"
    ],
    "Score": [
        knn_score,
        dt_score,
        lr_score,
        bnb_score,
        pa_score
    ]
}

score = pd.DataFrame(data1)

# =====
# STEP 10: DISPLAY RESULTS
# =====

print("\n=====")
print("CLASSIFICATION ALGORITHM SCORES")
print("=====")

print(score)

# =====
# STEP 11: DISPLAY ACCURACY IN PERCENTAGE
# =====

score["Accuracy (%)"] = (
    score["Score"] * 100
)

print("\n=====")
print("ACCURACY COMPARISON")
print("=====")

print(score)

# =====
# STEP 12: PREDICTIONS
# =====

knn_pred = knearestclassifier.predict(x_test)

dt_pred = decisiontree.predict(x_test)

lr_pred = logisticregression.predict(x_test)

bnb_pred = bernoulli_naiveBayes.predict(x_test)

pa_pred = passiveAggressive.predict(x_test)

```

```
# =====
# STEP 13: CLASSIFICATION REPORTS
# =====

print("\n=====")
print("KNN CLASSIFICATION REPORT")
print("=====")

print(
    classification_report(
        y_test,
        knn_pred
    )
)

print("\n=====")
print("DECISION TREE CLASSIFICATION REPORT")
print("=====")

print(
    classification_report(
        y_test,
        dt_pred
    )
)

print("\n=====")
print("LOGISTIC REGRESSION CLASSIFICATION REPORT")
print("=====")

print(
    classification_report(
        y_test,
        lr_pred
    )
)

print("\n=====")
print("BERNOULLI NAIVE BAYES CLASSIFICATION REPORT")
print("=====")

print(
    classification_report(
        y_test,
        bnb_pred
    )
)

print("\n=====")
print("PASSIVE AGGRESSIVE CLASSIFICATION REPORT")
print("=====")

print(
    classification_report(
        y_test,
        pa_pred
    )
)
```

First 5 rows of Iris Dataset:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Dataset Shape:
(150, 5)

Training Data:
(120, 4)

Testing Data:
(30, 4)

```
=====
CLASSIFICATION ALGORITHM SCORES
=====
Classification Algorithms      Score
```

```

0          KNN Classifier 1.000000
1    Decision Tree Classifier 1.000000
2      Logistic Regression 1.000000
3    Bernoulli Naive Bayes 0.300000
4 Passive Aggressive Classifier 0.633333

=====
ACCURACY COMPARISON
=====
Classification Algorithms    Score  Accuracy (%)
0          KNN Classifier 1.000000    100.000000
1    Decision Tree Classifier 1.000000    100.000000
2      Logistic Regression 1.000000    100.000000
3    Bernoulli Naive Bayes 0.300000    30.000000
4 Passive Aggressive Classifier 0.633333    63.333333

=====
KNN CLASSIFICATION REPORT
=====
precision    recall  f1-score   support

   setosa      1.00      1.00      1.00        10
  versicolor  1.00      1.00      1.00         9
   virginica  1.00      1.00      1.00        11

 accuracy      1.00      1.00      1.00        30
  macro avg      1.00      1.00      1.00        30
 weighted avg      1.00      1.00      1.00        30

=====
DECISION TREE CLASSIFICATION REPORT
=====
precision    recall  f1-score   support

   setosa      1.00      1.00      1.00        10

```

Experiment 17

```

# =====
# MOBILE PRICE RANGE PREDICTION
# USING LOGISTIC REGRESSION
# =====

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# =====
# STEP 1: CREATE DATASET IF CSV IS NOT FOUND
# =====

file_name = "mobile_prices.csv"

if not os.path.exists(file_name):

    print("mobile_prices.csv not found.")
    print("Creating sample mobile price dataset...")

    data_dict = {
        "battery_power": [
            842, 1021, 563, 615, 1821,
            1859, 1821, 1954, 1445, 509,
            825, 1524, 503, 775, 823,
            1134, 1308, 1073, 688, 1450,
            1022, 1113, 1830, 1134, 1990,
            1650, 1540, 900, 1200, 1350
        ],
        "blue": [
            0, 1, 1, 1, 1,
            0, 1, 1, 0, 1,
            0, 1, 1, 0, 1,
            0, 1, 0, 1, 0,
            1, 1, 0, 1, 0,
            1, 0, 1, 0, 1
        ]
    }

```

```
],  
  
"clock_speed": [  
    2.2, 0.5, 0.5, 2.5, 1.2,  
    0.5, 1.7, 0.5, 0.5, 0.6,  
    1.0, 0.5, 1.8, 1.4, 2.0,  
    1.5, 1.2, 1.8, 0.5, 2.0,  
    1.8, 2.0, 1.2, 1.5, 2.0,  
    1.0, 1.5, 2.0, 1.2, 1.8  
],  
  
"dual_sim": [  
    0, 1, 1, 0, 0,  
    0, 1, 1, 0, 1,  
    1, 1, 0, 1, 1,  
    0, 1, 0, 1, 1,  
    0, 1, 1, 0, 1,  
    1, 0, 1, 0, 1  
],  
  
"fc": [  
    1, 0, 2, 0, 13,  
    3, 0, 0, 0, 2,  
    4, 4, 2, 5, 1,  
    0, 2, 3, 1, 5,  
    0, 1, 4, 2, 1,  
    2, 3, 4, 1, 5  
],  
  
"four_g": [  
    0, 1, 1, 0, 1,  
    0, 1, 0, 1, 1,  
    1, 0, 1, 0, 1,  
    1, 0, 1, 0, 1,  
    0, 1, 1, 0, 1,  
    1, 0, 1, 0, 1  
],  
  
"int_memory": [  
    7, 53, 41, 10, 44,  
    22, 10, 24, 9, 4,  
    8, 16, 10, 32, 8,  
    4, 16, 32, 8, 16,  
    32, 64, 16, 8, 32,  
    64, 16, 8, 32, 64  
],  
  
"mobile_wt": [  
    188, 136, 145, 131, 141,  
    141, 149, 154, 146, 141,  
    196, 125, 96, 106, 88,  
    131, 81, 187, 108, 198,  
    150, 120, 160, 140, 170,  
    100, 180, 110, 155, 130  
],  
  
"n_cores": [  
    2, 3, 5, 6, 2,  
    3, 2, 8, 4, 3,  
    4, 8, 2, 6, 4,  
    8, 2, 4, 3, 6,  
    8, 4, 2, 6, 8,  
    4, 3, 8, 2, 6  
],  
  
"pc": [  
    2, 6, 6, 9, 7,  
    2, 0, 0, 14, 6,  
    3, 8, 5, 2, 1,  
    0, 3, 5, 2, 7,  
    8, 4, 6, 3, 5,  
    7, 2, 4, 8, 6  
],  
  
"px_height": [  
    20, 905, 1263, 1216, 1208,  
    1004, 381, 512, 386, 1137,  
    248, 775, 100, 670, 681,  
    360, 400, 950, 200, 800,  
    600, 720, 1100, 500, 900,  
    300, 850, 450, 700, 550  
],
```

```

"px_width": [
    756, 1988, 1716, 1786, 1212,
    1654, 1018, 1148, 836, 1224,
    874, 1962, 1990, 1264, 1280,
    1600, 1200, 1920, 1000, 1500,
    1800, 1300, 1600, 1100, 2000,
    900, 1700, 1200, 1900, 1400
],

"ram": [
    2549, 2631, 2603, 2769, 1411,
    1067, 3220, 700, 1099, 513,
    500, 1500, 1000, 1800, 1200,
    2200, 3000, 800, 1600, 2500,
    3200, 1800, 2800, 1000, 3500,
    1400, 2400, 900, 2000, 3000
],

"sc_h": [
    9, 17, 11, 16, 17,
    14, 8, 8, 12, 12,
    17, 16, 8, 16, 17,
    11, 12, 17, 9, 16,
    15, 14, 16, 10, 17,
    9, 15, 11, 16, 14
],

"sc_w": [
    7, 3, 2, 8, 3,
    4, 7, 8, 3, 5,
    1, 2, 5, 4, 3,
    6, 4, 7, 5, 3,
    8, 6, 7, 4, 8,
    5, 7, 3, 6, 4
],

"talk_time": [
    19, 7, 20, 7, 9,
    11, 13, 17, 20, 12,
    15, 13, 20, 18, 16,
    10, 18, 12, 8, 20,
    15, 17, 10, 19, 13,
    8, 16, 12, 18, 14
],

"touch_screen": [
    0, 1, 1, 0, 1,
    0, 1, 1, 1, 0,
    1, 0, 1, 0, 1,
    0, 1, 0, 1, 1,
    0, 1, 1, 0, 1,
    0, 1, 0, 1, 0
],

"wifi": [
    1, 0, 0, 0, 0,
    0, 1, 0, 1, 1,
    0, 1, 0, 1, 1,
    0, 1, 1, 0, 1,
    0, 1, 0, 1, 0,
    1, 0, 1, 0, 1
],

"price_range": [
    0, 1, 1, 2, 3,
    2, 3, 2, 1, 0,
    0, 2, 1, 2, 1,
    2, 3, 0, 1, 3,
    2, 1, 3, 2, 3,
    0, 2, 1, 3, 2
]
}

data = pd.DataFrame(data_dict)

data.to_csv(
    file_name,
    index=False
)

print("mobile_prices.csv created successfully!")

```

```

# =====
# STEP 2: IMPORT DATASET
# =====

data = pd.read_csv(
    file_name
)

# =====
# STEP 3: DISPLAY DATA
# =====

print("\n=====")
print("FIRST 5 ROWS")
print("=====")

print(
    data.head()
)

print("\nDataset Shape:")
print(
    data.shape
)

# =====
# STEP 4: CORRELATION HEATMAP
# =====

plt.figure(
    figsize=(12, 10)
)

sns.heatmap(
    data.corr(),
    annot=True,
    cmap="coolwarm",
    linecolor="white",
    linewidths=1
)

plt.title(
    "Mobile Price Dataset Correlation"
)

plt.show()

# =====
# STEP 5: DATA PREPARATION
# =====

x = data.iloc[
    :,
    :-1
].values

y = data.iloc[
    :,
    -1
].values

# =====
# STEP 6: STANDARDIZATION
# =====

scaler = StandardScaler()

x = scaler.fit_transform(
    x
)

# =====
# STEP 7: TRAIN TEST SPLIT
# =====

```



```

x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.20,
    random_state=0
)

# =====
# STEP 8: LOGISTIC REGRESSION
# =====

lreg = LogisticRegression(
    max_iter=1000
)

# =====
# STEP 9: TRAIN MODEL
# =====

lreg.fit(
    x_train,
    y_train
)

# =====
# STEP 10: PREDICTION
# =====

y_pred = lreg.predict(
    x_test
)

# =====
# STEP 11: ACCURACY
# =====

accuracy = (
    accuracy_score(
        y_test,
        y_pred
    ) * 100
)

print("\n=====")
print("LOGISTIC REGRESSION RESULTS")
print("=====")

print(
    "Accuracy of the Logistic Regression Model:",
    accuracy,
    "%"
)

# =====
# STEP 12: DISPLAY PREDICTIONS
# =====

print("\nPredicted Price Ranges:")

print(
    y_pred
)

# =====
# STEP 13: COUNT EACH PRICE RANGE
# =====

unique, counts = np.unique(
    y_pred,
    return_counts=True
)

price_range = np.asarray(

```

```
(unique, counts)
).T
```

```
print("\n=====")
print("PRICE RANGE PREDICTION COUNT")
print("=====")
```

```
print(
    price_range
)
```

```
# =====
# STEP 14: PRICE RANGE MEANING
# =====
```

```
print("\nPrice Range:")
print("0 = Low Cost")
print("1 = Medium Low Cost")
print("2 = Medium High Cost")
print("3 = High Cost")
```

```
print("\n=====")
```

```
mobile_prices.csv not found.
Creating sample mobile price dataset...
mobile_prices.csv created successfully!
```

```
=====
FIRST 5 ROWS
=====
```

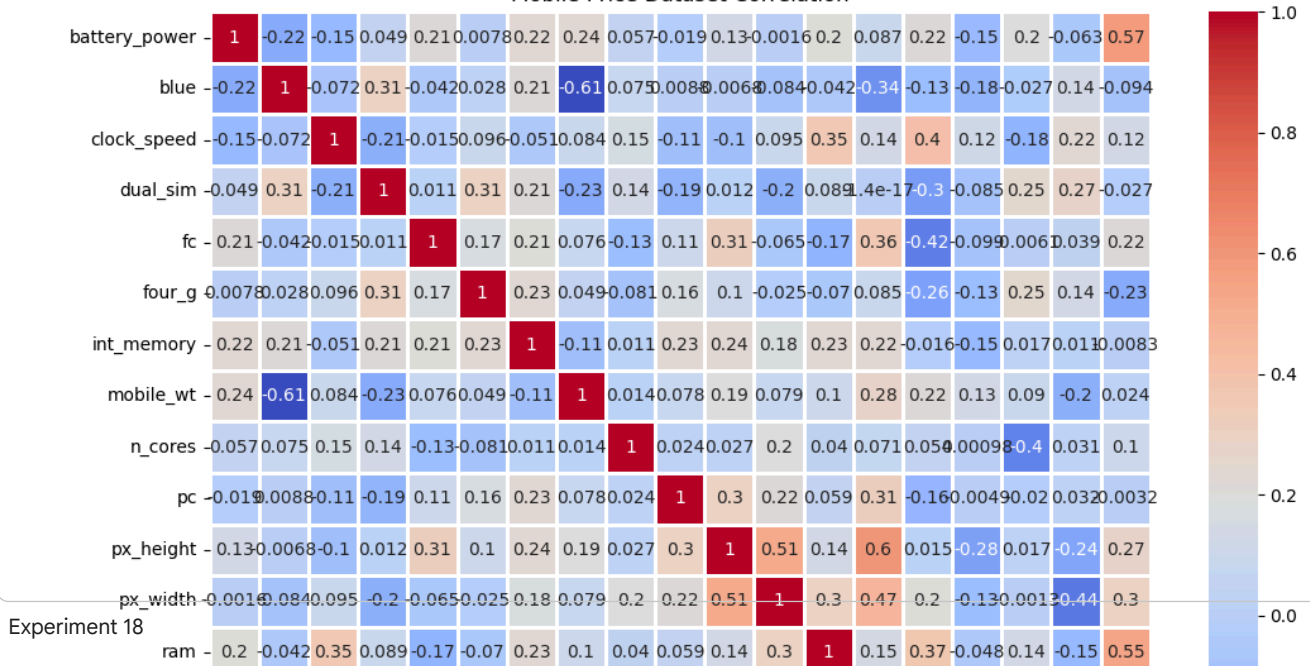
```
    battery_power  blue  clock_speed  dual_sim  fc  four_g  int_memory  \
0           842     0         2.2         0    1         0           7
1          1021     1         0.5         1    0         1          53
2           563     1         0.5         1    2         1          41
3           615     1         2.5         0    0         0          10
4          1821     1         1.2         0   13         1          44
```

```
    mobile_wt  n_cores  pc  px_height  px_width  ram  sc_h  sc_w  talk_time  \
0         188        2   2         20       756  2549    9    7          19
1         136        3   6         905      1988  2631   17    3           7
2         145        5   6        1263      1716  2603   11    2          20
3         131        6   9        1216      1786  2769   16    8           7
4         141        2   7        1208      1212  1411   17    3           9
```

```
    touch_screen  wifi  price_range
0              0     1             0
1              1     0             1
2              1     0             1
3              0     0             2
4              1     0             3
```

```
Dataset Shape:
(30, 19)
```

Mobile Price Dataset Correlation



```
# =====
# IRIS FLOWER CLASSIFICATION USING PERCEPTRON
```

```
# =====

from sklearn import datasets
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

# =====
# STEP 1: LOAD IRIS DATASET
# =====

iris = datasets.load_iris()

# =====
# STEP 2: SELECT FEATURES
# =====

# Selecting petal length and petal width
X = iris.data[:, [2, 3]]

# Target values
y = iris.target

# =====
# STEP 3: DISPLAY DATA
# =====

print("Feature Names:")
print(iris.feature_names)

print("\nSelected Features:")
print("Petal Length and Petal Width")

print("\nTarget Classes:")
print(iris.target_names)

# =====
# STEP 4: SPLIT DATASET
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1,
    stratify=y
)

print("\nTraining Data Shape:")
print(X_train.shape)

print("\nTesting Data Shape:")
print(X_test.shape)

# =====
# STEP 5: STANDARDIZE FEATURES
# =====

sc = StandardScaler()

sc.fit(X_train)

X_train_std = sc.transform(
    X_train
)

X_test_std = sc.transform(
    X_test
)

# =====
# STEP 6: CREATE PERCEPTRON MODEL
```

```

# =====

ppn = Perceptron(
    eta0=0.1,
    random_state=1
)

# =====
# STEP 7: TRAIN PERCEPTRON
# =====

ppn.fit(
    X_train_std,
    y_train
)

# =====
# STEP 8: PREDICT TEST DATA
# =====

y_pred = ppn.predict(
    X_test_std
)

# =====
# STEP 9: CALCULATE ACCURACY
# =====

accuracy = accuracy_score(
    y_test,
    y_pred
)

# =====
# STEP 10: DISPLAY RESULTS
# =====

print("\n=====")
print("PERCEPTRON CLASSIFICATION RESULTS")
print("=====")

print(
    "Accuracy using accuracy_score: %.3f"
    % accuracy
)

print(
    "Accuracy using model.score: %.3f"
    % ppn.score(
        X_test_std,
        y_test
    )
)

print("=====")

```

Feature Names:
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']

Selected Features:
Petal Length and Petal Width

Target Classes:
['setosa' 'versicolor' 'virginica']

Training Data Shape:
(105, 2)

Testing Data Shape:
(45, 2)

```

=====
PERCEPTRON CLASSIFICATION RESULTS
=====
Accuracy using accuracy_score: 0.978
Accuracy using model.score: 0.978
=====

```

```

# =====
# BREAST CANCER CLASSIFICATION
# USING GAUSSIAN NAIVE BAYES
# =====

import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score

# =====
# STEP 1: LOAD DATASET
# =====

try:
    dataset = pd.read_csv("breastcancer.csv")
    print("breastcancer.csv loaded successfully.")
except FileNotFoundError:
    print("breastcancer.csv not found.")
    print("Using built-in Breast Cancer dataset instead.")

    from sklearn.datasets import load_breast_cancer

    cancer = load_breast_cancer()

    X = cancer.data
    y = cancer.target

    dataset = pd.DataFrame(
        X,
        columns=cancer.feature_names
    )

    dataset["target"] = y

# =====
# STEP 2: SEPARATE FEATURES AND TARGET
# =====

X = dataset.iloc[:, :-1].values

y = dataset.iloc[:, -1].values

print("\nDataset Shape:")
print(X.shape)

print("\nFirst 5 Rows:")
print(dataset.head())

# =====
# STEP 3: SPLIT DATASET
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=0
)

print("\nTraining Data Shape:")
print(X_train.shape)

print("\nTesting Data Shape:")
print(X_test.shape)

# =====
# STEP 4: FEATURE SCALING
# =====

sc = StandardScaler()

```

```

X_train = sc.fit_transform(
    X_train
)

X_test = sc.transform(
    X_test
)

# =====
# STEP 5: CREATE GAUSSIAN NAIVE BAYES MODEL
# =====

classifier = GaussianNB()

# =====
# STEP 6: TRAIN MODEL
# =====

classifier.fit(
    X_train,
    y_train
)

# =====
# STEP 7: PREDICT TEST DATA
# =====

y_pred = classifier.predict(
    X_test
)

# =====
# STEP 8: CONFUSION MATRIX
# =====

cm = confusion_matrix(
    y_test,
    y_pred
)

print("\n=====")
print("CONFUSION MATRIX")
print("=====")

print(cm)

# =====
# STEP 9: CALCULATE ACCURACY
# =====

accuracy = accuracy_score(
    y_test,
    y_pred
)

print("\n=====")
print("GAUSSIAN NAIVE BAYES RESULTS")
print("=====")

print(
    "Accuracy:",
    accuracy
)

print(
    "Accuracy Percentage:",
    accuracy * 100,
    "%"
)

print("=====")

```

breastcancer.csv not found.
Using built-in Breast Cancer dataset instead.

Dataset Shape:
(569, 30)

First 5 Rows:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness \
0	17.99	10.38	122.80	1001.0	0.11840
1	20.57	17.77	132.90	1326.0	0.08474
2	19.69	21.25	130.00	1203.0	0.10960
3	11.42	20.38	77.58	386.1	0.14250
4	20.29	14.34	135.10	1297.0	0.10030

	mean compactness	mean concavity	mean concave points	mean symmetry \
0	0.27760	0.3001	0.14710	0.2419
1	0.07864	0.0869	0.07017	0.1812
2	0.15990	0.1974	0.12790	0.2069
3	0.28390	0.2414	0.10520	0.2597
4	0.13280	0.1980	0.10430	0.1809

	mean fractal dimension ...	worst texture	worst perimeter	worst area \
0	0.07871 ...	17.33	184.60	2019.0
1	0.05667 ...	23.41	158.80	1956.0
2	0.05999 ...	25.53	152.50	1709.0
3	0.09744 ...	26.50	98.87	567.7
4	0.05883 ...	16.67	152.20	1575.0

	worst smoothness	worst compactness	worst concavity	worst concave points \
0	0.1622	0.6656	0.7119	0.2654
1	0.1238	0.1866	0.2416	0.1860
2	0.1444	0.4245	0.4504	0.2430
3	0.2098	0.8663	0.6869	0.2575
4	0.1374	0.2050	0.4000	0.1625

	worst symmetry	worst fractal dimension	target
0	0.4601	0.11890	0
1	0.2750	0.08902	0
2	0.3613	0.08758	0
3	0.6638	0.17300	0
4	0.2364	0.07678	0

[5 rows x 31 columns]

Training Data Shape:
(426, 30)

Testing Data Shape:
(143, 30)

```
=====
CONFUSION MATRIX
=====
[[47  6]
 [ 6 84]]
=====
```

Experiment 20

```
# =====
# FUTURE SALES PREDICTION USING LINEAR REGRESSION
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error

# =====
# STEP 1: CREATE DATASET
# =====

data = pd.DataFrame({
    "TV": [
        230.1, 44.5, 17.2, 151.5, 180.8,
        8.7, 57.5, 120.2, 8.6, 199.8,
        66.1, 214.7, 23.8, 97.5, 204.1,
        195.4, 67.8, 281.4, 69.2, 147.3,
        218.4, 237.4, 13.2, 228.3, 62.3,
        262.9, 142.9, 240.1, 248.8, 70.6
    ],
    "Radio": [
```

```

        37.8, 39.3, 45.9, 41.3, 10.8,
        48.9, 32.8, 19.6, 2.1, 2.6,
        5.8, 24.0, 35.1, 7.6, 32.9,
        47.7, 36.6, 39.6, 20.5, 23.9,
        27.7, 5.1, 15.9, 16.9, 12.6,
        3.5, 29.3, 16.7, 27.1, 16.0
    ],

    "Newspaper": [
        69.2, 45.1, 69.3, 58.5, 58.4,
        75.0, 23.5, 11.6, 1.0, 21.2,
        24.2, 4.0, 65.9, 7.2, 46.0,
        52.9, 114.0, 55.8, 18.3, 19.1,
        53.4, 23.5, 49.6, 26.2, 18.3,
        31.6, 12.6, 22.9, 22.9, 40.8
    ],

    "Sales": [
        22.1, 10.4, 9.3, 18.5, 12.9,
        7.2, 11.8, 13.2, 4.8, 10.6,
        8.6, 17.4, 9.2, 9.7, 19.0,
        22.4, 12.5, 24.4, 11.3, 14.6,
        18.0, 12.5, 5.6, 15.5, 9.7,
        12.0, 15.0, 15.9, 18.9, 10.5
    ]
}

# =====
# STEP 2: DISPLAY DATA
# =====

print("First 5 rows:")
print(data.head())

print("\nSample 5 rows:")
print(data.sample(5))

print("\nMissing values:")
print(data.isnull().sum())

# =====
# STEP 3: CORRELATION
# =====

print("\nCorrelation:")
print(data.corr())

print("\nCorrelation with Sales:")
print(
    data.corr()["Sales"].sort_values(
        ascending=False
    )
)

# =====
# STEP 4: TV VS SALES
# =====

plt.figure(figsize=(7, 5))

plt.scatter(
    data["TV"],
    data["Sales"]
)

plt.xlabel("TV")
plt.ylabel("Sales")
plt.title("TV vs Sales")

plt.show()

# =====
# STEP 5: RADIO VS SALES
# =====

plt.figure(figsize=(7, 5))

plt.scatter(

```



```
        data["Radio"],
        data["Sales"]
    )

plt.xlabel("Radio")
plt.ylabel("Sales")
plt.title("Radio vs Sales")

plt.show()

# =====
# STEP 6: NEWSPAPER VS SALES
# =====

plt.figure(figsize=(7, 5))

plt.scatter(
    data["Newspaper"],
    data["Sales"]
)

plt.xlabel("Newspaper")
plt.ylabel("Sales")
plt.title("Newspaper vs Sales")

plt.show()

# =====
# STEP 7: PREPARE DATA
# =====

X = np.array(
    data[
        ["TV", "Radio", "Newspaper"]
    ]
)

y = np.array(
    data["Sales"]
)

# =====
# STEP 8: TRAIN TEST SPLIT
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# =====
# STEP 9: CREATE MODEL
# =====

model = LinearRegression()

# =====
# STEP 10: TRAIN MODEL
# =====

model.fit(
    X_train,
    y_train
)

# =====
# STEP 11: PREDICT TEST DATA
# =====

y_pred = model.predict(
    X_test
)
```

```
# =====
# STEP 12: MODEL SCORE
# =====

score = model.score(
    X_test,
    y_test
)

print("\n=====")
print("MODEL RESULT")
print("=====")

print(
    "Model Score:",
    score
)

# =====
# STEP 13: MEAN ABSOLUTE ERROR
# =====

error = mean_absolute_error(
    y_test,
    y_pred
)

print(
    "Mean Absolute Error:",
    error
)

# =====
# STEP 14: COEFFICIENTS
# =====

print("\n=====")
print("MODEL COEFFICIENTS")
print("=====")
```